MASTER THE PHYSICS OF MOTION: From first principles to real missions

Mastering the Math and Physics of
Space Orbits Simulation

Part I – Basic Astronomy and Celestial Mechanics

The Sun, Earth, Moon, Planets, Stars & Time

Richard L. Lachance, Ph.D.
SpaceOrbitLAB.com
RLLac@ULaval.ca

In collaboration with
ABB Space and Defence Systems

(The link)

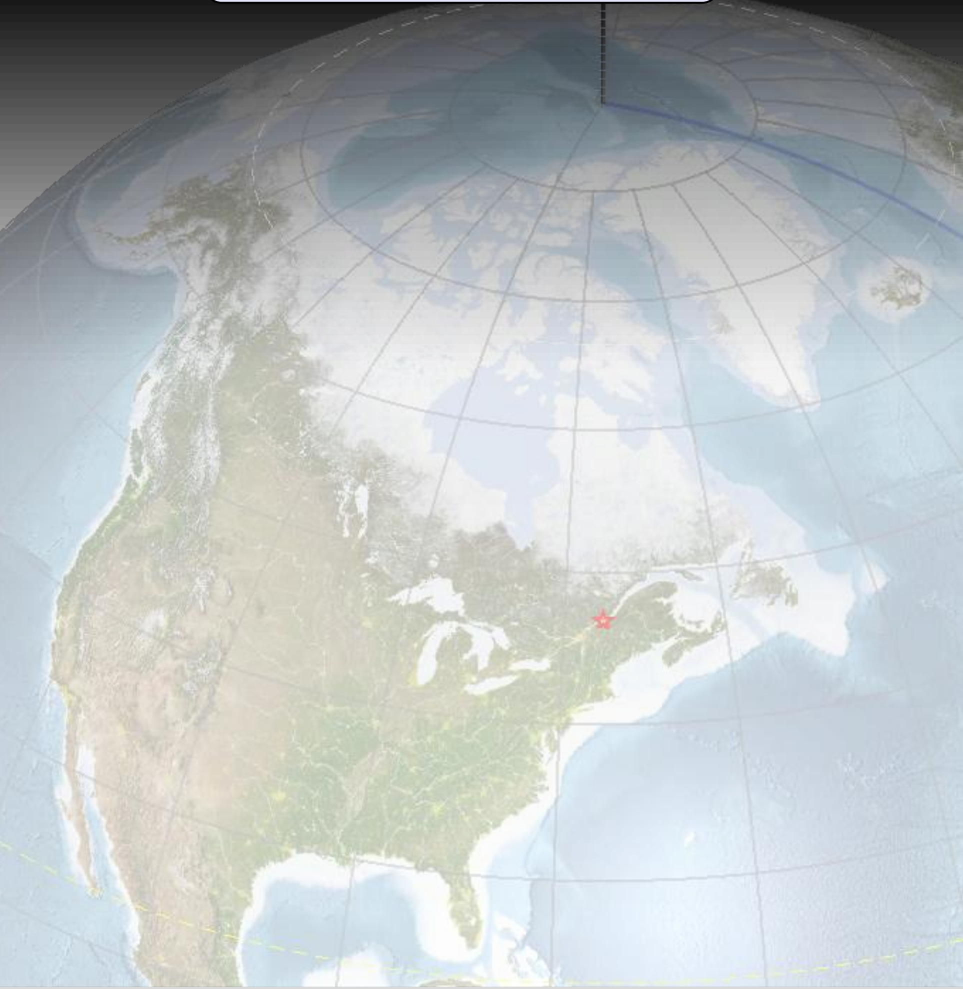
(function name)

Presentation plan
Course outline

i. Introduction
ii. History
iii. General concepts (Mathematics & Geometry)

Basic Astronomy: with emphasis on orbital mechanics

- **Sun-Earth-Moon** system
- **Planets, Asteroids & TNOs**
- Stars, Milky Way & Constellations
- **Time**



I– **Basic astronomy & Celestial Mechanics** – Course outline

I– **Basic astronomy & Celestial Mechanics** – Course outline

I– Basic astronomy & Celestial Mechanics – Course outline

Introduction

- Definitions
- History
- General concepts (Math & Geometry)

→ Astrodynamics will be presented in Part II

Angular notions

Definitions

- **Astronomy**: The scientific study of celestial objects and phenomena beyond Earth's atmosphere, encompassing their physical characteristics, chemical composition, radiation, motions, formation processes, and long-term evolution within the Universe.
  - It is also humanity's attempt to organize what we learn into a clear history of the universe, based on our most inventive theories from the distant past to the present moment.
- **Celestial mechanics**: A branch of astronomy concerned with the quantitative description and prediction of the motions of natural celestial bodies, governed primarily by gravitational interactions and formulated using Newtonian, perturbation, and relativistic mechanics.
  - Johannes Kepler's laws of planetary motion (1609–1619) and Newton's laws of motion (1687) are fundamental to it.
  - In the 18th century, powerful methods of mathematical analysis were generally successful in accounting for the observed motions of bodies in the solar system.
- **Astrodynamics**: The applied field derived from celestial mechanics that focuses on the motion, guidance, and control of artificial bodies in space, including spacecraft and satellites, with emphasis on orbit determination, maneuver planning, and mission design.
  - **Flight mechanics** deals with the motions of spacecraft under the influence of gravity, thrust, atmospheric drag, and other forces; it is used to calculate trajectories for ascent into space, achieving orbit, rendezvous, descent, and lunar and interplanetary flights.

Historical sketch from ancient times to Newton

- The historical development of celestial mechanics can be broadly divided into two complementary domains.
    1. First, there is the advancement of knowledge concerning the *formal and descriptive structure* of the heavens: the definition and measurement of natural time scales, the geometric organization of the celestial sphere, and the determination of planetary trajectories and orbital periods.
    2. Second, there is the pursuit of *physically accurate interpretations* of natural phenomena, encompassing the fundamental properties of force, matter, space, and time, and the laws governing their mutual relationships.
- Although these two lines of inquiry are conceptually distinct, they have not always been treated independently in practice. On the contrary, they have frequently been intertwined to such an extent that physical speculation has, at times, exerted an undue influence on purely kinematic or observational conclusions.
- While it is essential that these modes of investigation remain analytically distinct in the mind of the researcher, it is equally important that they be used in conjunction, serving as mutual constraints and consistency checks.

General Concepts
Some required math and geometry

- Notation, Symbols & Alphabet
- Units, Physical & Astrodynamics Constants
- Angular notions, Solid angle
- Trigonometric functions
- Vectors, Rotation matrices
- Statistics
- General programming notes
- Validation & pitfalls
- Selection of a programming language
- Some typography
- Use of Artificial Intelligence (AI)

Notation: symbols & alphabet

Greek letters & special symbols

α	alpha
β	beta
γ	gamma
δ	delta
ε	epsilon
θ	theta
λ	lambda
μ	mu
ν	nu
π	pi (≈3.1416…)
ρ	rho
σ	sigma
ϕ φ	phi
Φ	Phi (uppercase)
∠	angle
ø	diameter

Alternate forms →

ψ	psi (lowercase)
Ψ	Psi (uppercase)
ω	omega (lowercase)
Ω	Omega (uppercase)
☉	Sun
⊕	Earth
☾	Moon
☆	Sidereal
♈	First Point of Aries (`1stA`) – *Vernal Equinox* (ϒ-ϒ)
♋	Summer Solstice (Cancer)
☊ ☋	Ascending / Descending node
◊	Epoch
ℓ	Ellipse semi-parameter; focal parameter (p).
≡	Defined as / Identity, e.g.: $\sin^2(x) + \cos^2(x) \equiv 1$; $\sin(90°) \equiv 1$
	(also congruence in modular arithmetic: $22 \equiv 2 \pmod{10}$)

Winter solstice → ❄ ♈ ← **Spring equinox**
Summer solstice → ☀ ♎ ← Autumn equinox

Special symbols (continued)

' = dumb quote (typewriter or internet apostrophe) → avoid

’ = punctuation apostrophe or right single quote (e.g. it’s natural)

” = English quote (e.g. “this is OK” and "*this is not*") → français: « école »

$'$ = prime symbol (e.g. $\theta' = f(\theta)$, $1'$ ≡ one arc-minute)

- feet or minutes
- transformed coordinates
- conjugate points
- derivatives
- the complement F' of a set F
- an alternate notation for transpose

$''$ = double prime (e.g. $1''$ ≡ one arc-second, $6'3''$ six feet and 3 inches)

- inches or seconds
- arc seconds

Notation: Astrodynamics

$\vec{r}$	Position vector, $\lvert\vec{r}\rvert = \rho$
$\vec{v}$	Velocity vector, $\lvert\vec{v}\rvert$ ≡ speed
$\vec{h}$	Specific Angular Momentum vector, $\lvert\vec{h}\rvert = h$
$\vec{n}$	Ascending Node vector, $\lvert\vec{n}\rvert = r_P$
$\vec{e}$	Eccentricity vector, $\lvert\vec{e}\rvert = e$
μ	Gravitational constant
ε	Energy / also Earth's obliquity ϵ
e	Eccentricity
a	Semi-Major Axis
i	Inclination
ω	Argument of Perigee / angular rotation speed
ϖ	Longitude of Perihelion (used in planetary astronomy)
Ω	RA of Ascending Node (RAAN) / also Solid Angle
ν	True Anomaly (also θ)
ρ	Orbit radius length
ϵ	Earth obliquity of the ecliptic (23.44°)

α	Right Ascension (RA) / Azimuth (Azim)
δ	Declination (Dec) / Elevation (Elev)
λ	Longitude
ϕ	Geocentric latitude
φ	Geodetic latitude (sub-satellite)
ϕ	Flight Path Angle (FPA)
γ	Zenith Angle
β	Launch Azimuth (LA) angle
$\beta_\odot$	Sun Beta Angle
ψ	Orbit heading angle
ξ, ζ	Complementary angles
σ	Wavenumber
λ	Wavelength
β	Fraction of diffuse light
ψ	phase angle
Φ	phase integral / radiant flux
Ψ	reflected radiance factor

Radiometry

Notation (continued)

α Thermal Absorptivity (material specific) / Albedo

ε Thermal Emissivity (dependent on material and temperature)

Θ Angle Between the incident Sun vector and the affected plate surface normal (→ with Thermal calculations)

σ Stefan Boltzmann Constant / Standard deviation

Units

- Système International (SI)
- Binary or decimal prefixes
- Derived units

International System of Units – SI

- The International System of Units, internationally known by the abbreviation SI (from *Système international d'unités*), is the modern form of the metric system and the world's most widely used system of measurement.
  - It is the only system of measurement with official status in nearly every country in the world, employed in science, technology, industry, and everyday commerce.
  - The SI system is coordinated by the BIPM – Bureau international des poids et mesures
- The SI comprises a coherent system of units of measurement starting with seven base units:

The seven SI base units

Symbol	Name	Quantity
s	second	time
m	meter	length
kg	kilogram	mass
A	ampere	electric current
K	kelvin	thermodynamic temperature
mol	mole	amount of substance
cd L	Candela Radiance	luminous intensity Radiant power [W/(m²·sr)]

← derived unit used in Radiometry

SI (metric) prefixes

Symbol	Prefix	English	Français (long scale)
10^{24} = Y	yotta	septillion	quadrillion
10^{21} = Z	zetta	sextillion	trilliard
10^{18} = E	exa	quintillion	trillion, mille billiards
10^{15} = P	peta	quadrillion	billiard, mille billions
10^{12} = T	tera	trillion	billion, mille milliards
10^{9} = G	**giga**	billion	milliard
10^{6} = M	**mega**	million	million
10^{3} = k	**kilo**	thousand	mille
10^{2} = h	hecto	hundred	cent
10^{1} = da	deca	ten	dix

10^{-1} = d	deci	1/10
10^{-2} = c	**centi**	1/100th
10^{-3} = m	**milli**	thousandth
10^{-6} = µ	**micro**	millionth
10^{-9} = n	**nano**	billionth
10^{-12} = p	**pico**	
10^{-15} = f	femto	
10^{-18} = a	atto	
10^{-21} = z	zepto	
10^{-24} = y	yocto	

Zetta

- 1 zettaliter (ZL) = 1×10^{21} L (uppercase L preferred)
- The total volume of Earth's oceans is ≈ 1.386 ZL, representing over 97% of all water on Earth
- This massive volume includes the Pacific, Atlantic, Indian, Southern, and Arctic Oceans.
  → see Section Ocean Tides

Sources:

NOAA: How much water is in the ocean?
https://oceanservice.noaa.gov/facts/oceanwater.html

Earth Doesn't Actually Have The Most Water in The Solar System
www.sciencealert.com/earth-water-solar-system-comparison-moons-have-oceans-ice

World	Liquid water volume	World volume	% water
Enceladus	0.01 ZL	0.07 ZL	68%
Triton	0.03 ZL	10.35 ZL	65%
Dione	0.14 ZL	0.74 ZL	63%
Pluto	1.0 ZL	7.01 ZL	62%
Earth	1.335 ZL	1,083.21 ZL	0.12%
Europa	2.6 ZL	16.06 ZL	18%
Callisto	5.3 ZL	58.63 ZL	33%
Titan	18.6 ZL	71.60 ZL	44%
Ganymede	35.4 ZL	76.29 ZL	69%

Earth has a surprisingly small amount of water compared to other worlds in the Solar System

Each measurement shows the volume of the world and its stores of liquid water (ice not included).

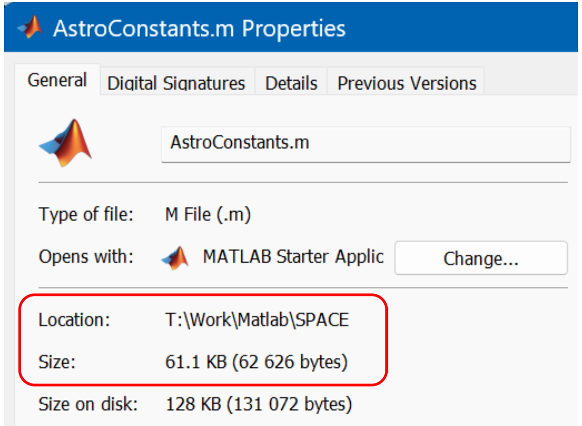
1 zettaliter (ZL) = 1,000,000,000,000,000,000,000 liters

Beware of binary prefixes

- In computing, a custom arose of using the metric prefixes to specify powers of 2.
  - For example, a kilobit was sometimes 2^{10} = 1024 bits instead of 1000 bits.
  - This practice led, and still leads, to considerable confusion because binary systems use powers of two, and the SI metric system uses powers of ten.
- The **megabyte** is a multiple of the unit byte for digital information (https://en.wikipedia.org/wiki/Megabyte)
  - Its recommended unit symbol is MB (as opposed to Mb for mega-bit)
  - The unit prefix mega is a multiplier of 1000000 (10^6) in the International System of Units (SI).
  - Therefore, one megabyte is *one million bytes* of information.
  - This definition has been incorporated into the International System of Quantities (ISQ).

→ Always specify what is used: bit or byte (≡8 bits), base 2 → base 10 , or use the “bibi” units…

1 KB = 2^{10} = 1024 bytes →



Classic source of confusion: Base 2 or 10 ?

The confusion comes from the difference between SI definitions (decimal) and binary usage (power of 2) in computing

1. SI (Decimal) Definition

- In the International System of Units (SI), prefixes like *kilo*, *mega*, *giga* follow powers of 10.
- This is the definition recognized by standards bodies (ISO, IEC, NIST, SI).
- Storage manufacturers (hard drives, SSDs, USB sticks) follow this definition, which is why a “500 GB drive” actually means 500×10^9 bytes:
  - Instead of showing 500 GB, Windows shows 465 GB (actually 465 GiB). → 500 * 1e9/2^30 = 465.6
  - The “missing” ≈35 GB isn’t lost; it’s just a mismatch between decimal GB (manufacturer) and binary GB (Windows) definitions → *hidden selling point.*

2. Binary (Power of 2) Definition

- Computers operate in binary, so memory sizes often fit better into powers of 2
  - 1 mebibyte (MiB) = 2^{20} = 1,048,576 bytes, 1 gibibyte (GiB) = 2^{30} = 1,073,741,824 bytes
- Historically, programmers (and operating systems like Windows) used “KB, MB, GB” as shorthand for these binary multiples

Classic source of confusion: Base 2 or 10 ?

- Windows still uses the **binary interpretation** in file sizes and memory reporting:
  - When Windows says “1 MB,” it really means 1 MiB = 1,048,576 bytes.
- Windows reports sizes in **GiB** (gibibytes), but labels them as “GB”
- This persisted for backward compatibility, since early computing culture adopted the binary meaning before the SI standard was strictly enforced.
- Both are correct, but in different contexts:
  - SI standard (MB ≡ 10^6 bytes): used by storage device manufacturers and in scientific/engineering contexts.
  - Binary convention (MB ≡ 2^{20} bytes): still used by many operating systems (especially Windows) for memory and file sizes.
- To reduce ambiguity, the IEC introduced new binary prefixes in 1998:
  - KiB, MiB, GiB, etc.But adoption is mixed — Linux and macOS often use them properly, while Windows sticks with the old usage.

→ Wikipedia follows the official SI definition (1 MB = 10^6 bytes), but Windows follows the traditional binary definition (1 MB = 2^{20} bytes). That’s why you see the difference. → *be careful*!

Derived units

- The SI assigns special names to 22 derived units, which includes two dimensionless derived units, the radian [rad] and steradian [sr]
- The seven base units and the 22 coherent derived units with special names and symbols may be used in combination to express other coherent derived units

Name	Symbol	Quantity	Equivalents	SI based
hertz	Hz	frequency	1/s	s^{-1}
radian	rad	angle	m/m	1
steradian	sr	solid angle	m^2/m^2	1
newton	N	force, weight	$kg{\cdot}m/s^2$	$kg{\cdot}m{\cdot}s^{-2}$
pascal	Pa	pressure, stress	N/m^2	$kg{\cdot}m^{-1}{\cdot}s^{-2}$
joule	J	energy, work, heat	m·N, C·V, W·s	$kg{\cdot}m^2{\cdot}s^{-2}$
watt	W	power, radiant flux	J/s, V·A	$kg{\cdot}m^2{\cdot}s^{-3}$
coulomb	C	Electric charge	s·A, F·V	A·s
volt	V	voltage, electrical potential difference	W/A, J/C	$kg{\cdot}m^2{\cdot}s^{-3}{\cdot}A^{-1}$
farad	F	electrical capacitance	C/V, s/Ω	$kg^{-1}{\cdot}m^{-2}{\cdot}s^4{\cdot}A^2$
ohm	Ω	electrical resistance	1/S, V/A	$kg{\cdot}m^2{\cdot}s^{-3}{\cdot}A^{-2}$
tesla	T	magnetic induction/ flux density	$V{\cdot}s/m^2$, N/(A·m)	$kg{\cdot}s^{-2}{\cdot}A^{-1}$
degree Celsius	°C	temperature relative to 273.15 K	K	K
lumen	lm	luminous flux	cd·sr	cd
lux	lx	illuminance	lm/m^2	$cd{\cdot}m^{-2}$
Irradiance	E	Irradiance, (heat) flux density	W/m^2	
Radiance	L	Radiance	$W/(m^2{\cdot}sr)$	

See details and use in second part of the → SOL course Part III: Remote Sensing from Space

Le système métrique

- Le Système Métrique → Origine, définitions, et pourquoi les États Unis refusent d'utiliser le système métrique
  - https://youtu.be/oueNksBpXms?si=dVzFL9kn6HRRLrNe

- Système impérial?

Base quantity	Symbol	SI unit	USC unit
Mass	M (or m)	kilogram [kg]	slug (≠ pound)
Length	L (or l)	meter [m]	foot (ft)
Time	t	Second [s]	second (s)
Temperature	T	Kelvin [K]	Rankine (R or R)

Units

- **Force**: Newton [N ≡ kg·m/s^2]
  - 1 N is the force required to accelerate an object of 1 kg at an acceleration of 1 m/s^2
  - Pressure: Pascal [N/m^2] per surface area
- **Energy**: Joule [J ≡ kg·m^2/s^2 ≡ N·m ≡ W·s]
  - 1 J is the energy required for lifting an apple 1 m high
- **Power**: Watt [W ≡ J/s]
  - The quantity of energy deployed by unit of time
  - Irradiance: [W/m^2] per surface area
- Electric charge: Coulomb [C]
  - Electron charge = $1.60217662 \times 10^{-19}$ [C/e$^-$]
  - The amount of electricity that a 1 A current carries in 1 s
- Electric current: Ampere [A ≡ C/s]
- Responsivity: A/W = C/J (for a detector)

Mass & Weight

- **Mass** and weight are related concepts but have distinct meanings. Mass is a measure of the amount of matter in an object; it is an intrinsic property related to inertia (not gravity) and so independent of its location in the universe. The unit of mass in SI is the kilogram [kg]
- **Weight** is the force exerted on an object under gravity. It depends on the object's mass and the acceleration under gravity at its specific location in the universe. The unit of weight is the Newton [N]
- The key difference between mass and weight lies in their definitions and how they are measured.
  - Mass is a scalar quantity, while weight is a force (a vector) that can change based on the gravitational field strength
- Newton's second law relates mass, gravitational acceleration, and weight as: $W = M\,\mathrm{g}$, or $W\vec{k} = M(-\mathrm{g}\,\vec{k})$ in vector form, where $\vec{k}$ is the unit normal vector in the upward vertical direction, and the minus sign denotes that weight is downward, directed toward the center of the Earth.

  On the surface of the Earth at mean sea level (MSL), the magnitude of g_0 is ≈ 9.81 m/s^2.
- An object weighs less on the Moon than on Earth because the Moon's gravitational force is only ≈1/5 that on Earth

→ In everyday use, "weight" is often measured in kg, the latter being an anomaly of the SI system that, if used, must be reconciled for engineering calculations by multiplying by the value of g (or g_0 if so defined).

Power – Watt

- A watt [W] is a unit of power, which is energy used or given off per unit time.
- It is measured in *joules per second* [J/s].
- We know that it is not just how much energy is expended, but how long it takes to do it
  - Burning 10 Calories in 10 minutes requires a very different kind of exercise than burning those 10 Calories in an hour.
- Watts tell you the rate at which energy is being used
  - A 100-W bulb uses 100 J of energy every second.
- And how big is a joule?
  - Lifting an 100 g apple 1 m upward requires ≈ 1 J of work
  - Kicking a soccer ball – 10 to 100 J of energy
  - Food energy – A single calorie (from food) equals about 4,184 J
  - It takes about 4,200 J to heat 1 liter of water by 1°C.
  - A fully charged AA battery stores ≈10,000 J of energy
  - A small car moving at 100 km/h has about 2–3 M J of kinetic energy
  - A ball leaving the bat at 110 MPH (49 m/s) carries ≈175 Joules of energy (average professional hit)
  - Impact energy of a small handgun bullet ≈ 300 J

Horsepower

- One imperial horsepower lifts 550 pounds (250 kg) by 1 foot (30 cm) in 1 second
  - hp = 550 [ft-lb s^{-1}] = **745.7 [W]**
  - https://en.wikipedia.org/wiki/Horsepower

What is a unit of horsepower?

The mechanical output of an engine is often measured in "horsepower," which is given the unit symbol "hp" and not "HP" or "Hp" or other such variation. This unit is attributed to the 18th-century Scottish engineer James Watt, whose work on steam engines was fundamental to the changes in society brought about by the Industrial Revolution. James Watt wanted to market his engines by comparing them to the equivalent power produced by a horse, which was a tangible measure to the general public and farmers. Watt did many experiments and determined that a typical Scottish farm horse could, on average, steadily lift a 600 lb weight through a simple pulley system for an average distance of 63.9 ft in an average of 69 seconds; averaging here was important. Therefore, the work done by an average horse is

$$\text{Work} = F\,d = 600 \times 63.9 = 38,340 \text{ ft-lb} \tag{2}$$

Power is the rate of doing work, so the work per unit time is

$$\text{Power} = \frac{\text{Work}}{\text{Time}} = \frac{38,340}{69} = 555.65 \text{ ft-lb s}^{-1} \tag{3}$$

James Watt settled on the average result that one horsepower (hp) = 550 ft-lb s^{-1}. This measurement unit of hp made much more sense to farmers and others using horses. For example, for a steam engine of power 10 hp, the purchaser knew precisely what they were buying – an engine with the power equivalent to 10 horses. Marketing genius! Modern usage of horsepower is prevalent in the automotive and industrial fields to describe the power output of engines, motors, and other machinery. The horsepower unit caught on and is still widely used in many countries, even those using SI units exclusively.

https://eaglepubs.erau.edu/introductiontoaerospaceflightvehicles/chapter/units-conversion-factors/

Handling units

- Always properly identify and handle units associated with each variable, either in name or in comments
- This is a good way to proceed, helping to correctly build structures and coherent scientific code
  - Helps to understand progress in equations development
  - Avoiding errors by combining incompatible units
  - → Trust me: *this saves lives*!

```
Angle_rad = 0.123;             % [rad]      ← unit in variable name to avoid any misunderstanding
Angle_deg = DEG(angle_rad);    % [°]
Eccentricity = 0.003;          % [n.u.]     ← for no units
LightSpeed_ms = 299792458;     % [m/s]      ← SI units recommended
LightSpeed_kms = 3e5;          % [km/s]     ← if required
```

- In PYTHON, there is the ability to assign units to variables
  - pip install numericalunits (TBC)

Useful conversion functions

```
function kg = lb_to_kg(lb)
% Converts pounds to kg
    kg = lb * 0.453592;  % lb_to_kg(2.2) ≈ 1 kg
end

function m = inch_to_m(in)
% Converts inches to meters
    m = in * 25.4/1000;  % inch_to_m(12) * 100 ≈ 30 cm
end

function m = foot_to_m(ft)
% Converts inches to meters
    m = (ft * 0.3048);  % foot_to_m(3.3) ≈ 1 m
end

function km = mile_to_km(mile)
% Converts miles to km
    km = mile * 1.60934;  % mile_to_km(100) ≈ 161 m
end
```

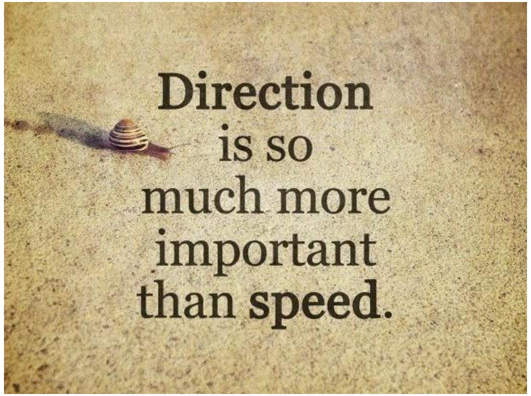
Various notes

- When writing, be clear as possible:

```
% Wavelength at peak of Sun's spectral exitance in wavelength = 502.25 nm
lambda_peak = 5.0225e-7;  % [m] mathematically correct, but can be misleading
lambda_peak = 502.25e-9;  % [m]  ← Preferred
lambda_peak_nm = 502.25;  % [nm] also
```

Speed vs velocity

- The primary difference is that speed is a scalar quantity, representing only the magnitude (how fast) of an object's motion, while velocity is a vector quantity, including both the magnitude (speed) and the direction of the motion.
- Speed is calculated using distance over time, whereas velocity is calculated using displacement (change in position) over time.
  - For example, a car moving at 100 km/h has a speed of 100 km/h, but its velocity would be 100 km/h north.



Physical & Astrodynamics Constants

Fundamental Physical Constants

Quantity	Symbol	Value	Units	Variable name
Fundamental Physical Constants				
Universal Gravitational Constant	G	6.67430e-11 ±0.00015e-11	N·m²/kg² ≡ m³/(kg·s²)	`G`
Planck constant	h	6.6261E-34	J·s ≡ kg·m²/s	`Planck_cte`
Speed of light (in vacuum) Speed of electromagnetic radiation	c	299792458	m/s	`LightSpeed_m`
Electron charge (1 Ampere = 1 C/s)	e	1.60217662E-19	C/e⁻	`Electron_charge`
Boltzmann's constant Proportionality factor relating the average relative thermal energy of particles in a gas with the thermodynamic temperature of the gas.	k or k_B	1.3806503e-23	J/K	`Boltzmann_cte`
Stefan-Boltzmann's constant Constant relating the intensity of the thermal radiation emitted by matter in terms of that matter's temperature $\sigma = (2\pi^5k^4) / (15\ h^3c^2)$	σ	5.670374419e-08	W/(m²·K⁴)	`Stefan_Boltzmann_cte`
Wien wavelength displacement law constant	b_l	2.8978e-3	m·K	`Wien_cte`
Light-year distance (Julian year)	ly	9460730472581	km	`LightYear_km`
Number of light-years in a parsec		3.26156	n.u.	`ly_in_parsec`
Zero point of apparent bolometric magnitude scale	$Irr_{0_{bolo}}$	2.518021002e-8	W/m²	`Irr0_bolo`

→ See how constants in black are computed in DefineConstants and AstroConstants

Fundamental Physical Constants (continued)

Quantity	Symbol	Value	Units	Variable name
Atomic particles Mean Molar Mass (or Molar Weight)				
Atomic mass unit	u	1.66053873e-27	kg	u
Hydrogen	H	1.00784	kg	H_atomic_mass
Oxygen	O	15.9994	kg	O_atomic_mass
Carbon	C	12.0107	kg	C_atomic_mass
Nitrogen	N	14.0067	kg	N_atomic_mass
Argon	Ar	39.948	kg	Ar_atomic_mass
Gas & Atmosphere characteristics				
Avogadro's Number Number of units in one mole of any substance (defined as its molecular weight in grams)	N_A	6.0221e+23	mol^{-1}	Avogadro
Gas constant ($\equiv A{\cdot}k_B$)	R	8.314462618	J/(mol·K)	R_gas_constant
Zero Celsius		273.15	K	Zero_C
Atmospheric pressure	P_0	101325	Pa	Atm_pres_Pa
Air dry mean molar mass		28.96354	g/mol	Air_MolMass
Dry air molar weight	M_a	0.028963539616	kg/mol	M_a
Air Molecular Mass ($\equiv M_a$/Avogadro)		4.8095e-26	kg/molecule	Air_MolecularMass
Average air temperature at sea level ≈ 15°C		288.15	K	T_air_K
Air density at sea level and 15°C		1.2249489	kg/m^3	Air_density_kg_m3
Standard molar volume of ideal gas	V_m	22.413838	L/mol	Vm

Spaceflight constants

Quantity	Symbol	Value	Units	Variable name	
Earth					
Geocentric gravitational constant, Standard Earth Gravitational Parameter (GM)	$\mu_\oplus$	398600.4418 ±0.0008	km³/s²	`Earth.mu`	
Earth mass	$M_\oplus$	5.9723+24 ±1e-4	kg	`Earth.mass_kg`	$\equiv \mu/G$
Orbit eccentricity	e	0.016698	n.u.	`Earth.e`	
Orbit semi-major axis	a	149598023	km	`Earth.a`	
Astronomical Unit	AU	149597870.7	km	`Earth.AU`	
Obliquity of the ecliptic at J2000.0	ϵ_{J2000}	23.4392794	degree	`Earth.e_J2000`	
Equatorial radius at sea level	r_1 ; $r_\oplus$	6378.137	km	`Earth.r1_km`	
Polar radius (at sea level)	r_2	6356.7523	km	`Earth.r2_km`	
Average radius	r	6371.00877	km	`Earth.radius_km`	$\equiv (2\,r_1 + r_2)/3$
Circumference		40075	km	`Earth.circ_km`	$\equiv 2\pi\, r_1$
Eccentricity of the Earth's oblate ellipsoidal shape		0.081819	n.u.	`Earth.ecc`	$\equiv \sqrt{1-(r_2/r_1)^2}$
Earth's "flattening" (→ should be called *oblateness*)		0.00335281	n.u.	`Earth.f`	$\equiv 1 - r_2/r_1$ $\approx 1/298.25702$

Earth gravitational acceleration
1) Standard gravity used for general physics, atmospheric models, and engineering
2) Acceleration due to gravity at Earth's equatorial radius; for general Orbital Mechanics ($a = \mu/r^2$)
	g_0
	
	g
		9.80665
		
		9.797643222
			m/s²	`Earth.g0`
				
				`Earth.g`	

Spaceflight constants (continued)

Quantity	Symbol	Value	Units	Variable name
Sun				
Sun Gravitational Parameter ($G\,M_\odot$) for small satellites orbiting the Sun; ≠ G ($M_\odot$+$M_\oplus$)	$\mu_\odot$	132712440042	km³/s²	`Sun.mu`
Sun mass	$M_\odot$	1.988416e30	kg	`Sun.mass_kg`
Equatorial radius (IAU2015)	$r_\odot$	695700	km	`Sun.radius_km`
Flattening (near-perfect spherical shape)		0.00005	n.u.	
Solid angle under which the Sun appears	$\Omega_\odot$	6.79489e-05	sr	`Sun.Omega`
Angular diameter (Average over one year)		0.5329	degree	`Sun.Angular_diameter`
Solar luminosity	$L_\odot$	3.828e26	W	`Sun.luminosity`
Solar constant (at 1 AU)		1361.6	W/m²	`Sun.solar_cte`
Average surface temperature (all wavelengths; fit within visible range	$T_\odot$	5777 5875	K	`Sun.T_K` `Sun.T_K_VIS`
Wavelength of peak in solar spectrum	l_{peak}	502.25e-9	m	`Sun.I_peak`
Sun density		1.4098	g/cm³	`Sun.density`
Sun-Earth average distance → 1 AU (see above)				

Astronomical constants values
https://nssdc.gsfc.nasa.gov/planetary/factsheet/Earthfact.html
https://ssd.jpl.nasa.gov/astro_par.html

Spaceflight constants (continued)

Quantity	Symbol	Value	Units	Variable name
Moon				
Moon Gravitational Parameter ($GM_☾$)	$\mu_☾$	4904.0575	km³/s²	Moon.mu ← for small satellites orbiting the Moon
Moon mass	$M_☾$	7.34767309e22	kg	Moon.mass_kg
Orbit eccentricity	$e_☾$	0.0549006	n.u.	Moon.e
Orbit semimajor-axis (geocentric Lunar orbit)	$a_☾$	384399	km	Moon.a
Minimum distance to Earth (perigee) Maximum distance to Earth (apogee) Average distance* in time $\langle r \rangle = a(1 + e^2/2)$		356371 406720 384978	km	Moon.min_dist Moon.max_dist Moon.mean_dist
Orbital inclination wrt ecliptic	$i_☾$	5.145396	degree	Moon.i
Mean inclination of lunar equator to ecliptic		1.543	degree	Moon.i_
Moon radius	$r_☾$	1737.4	km	Moon.radius_km
Solid angle under which the Moon appears	$\Omega_☾$	6.63596e-05	sr	Moon.Omega
Angular diameter (average over one year)		0.51793	degree	Moon.Angular_diameter
Earth-Moon barycenter offset wrt to Earth's center		4671.855	km	Earth.barycenter_offset
Sidereal period Tropical period Draconic period Anomalistic period Synodic period		27.321661 27.321582 27.212222 27.554551 29.5305889	days	Moon.Sidereal_period Moon.Tropical_period Moon.Draconic_period Moon.Anomalistic_period Moon.Synodic_period

*"Average Distance" → SOL Part II

Simplifying units? Be careful

Radiometry

- Irradiance E: [W/cm^2] , i.e., power per surface area
- Spectral Irradiance: [W/(cm^2·cm^{-1})], i.e., irradiance per wavenumber unit
  (Note: the two *cm* are not the same: the first is a geometrical, the second is related to spatial frequency)

→ DO NOT SIMPLIFY to [W/cm] ! which would lose its physical sense and become meaningless and confusing…

Spectroscopy

- Molecular transition intensity S: [cm^{-1}/(molec/cm^2)] representing absorption cross-section per unit column density
  - The *mass path* gives the number of molecules per cross-sectional area in the path [molecules/cm^2]

→ DO NOT SIPLIFY to [cm/molec] ! which would lose its physical sense

→ Also, DO NOT WRITE: [cm^{-1}/(mol·cm^{-2})] because a *mole** and *molecule* are not the same

nor write: [cm^{-1} molec^{-1} cm^2], or [cm^{-1}/(molec cm^{-2})], like sometimes seen, which is confusing

* The mole, abbreviated mol, is an SI unit which measures the number of particles in a specific substance.
One mole is equal to $6.02214179 \times 10^{23}$ atoms, or other elementary units such as molecules.

Quiz

Question: in what unit are we really talking about?

- House electric consumption: "kW·h"
  → [(J/s)·s] ≡ J energy consumption
- Batterie charge expressed in "mA·h"
  → [(C/s)·s] ≡ C total charge

… like stating distances in [(km/h)·min]

Exercise

- Give the units and identify whether the following quantities are scalars or vectors:

    a. Position
    b. Distance
    c. Torque
    d. Force
    e. Energy
    f. Power
    g. Momentum
    h. Speed
    i. Velocity
    j. Mass
    k. Weight

Exercise

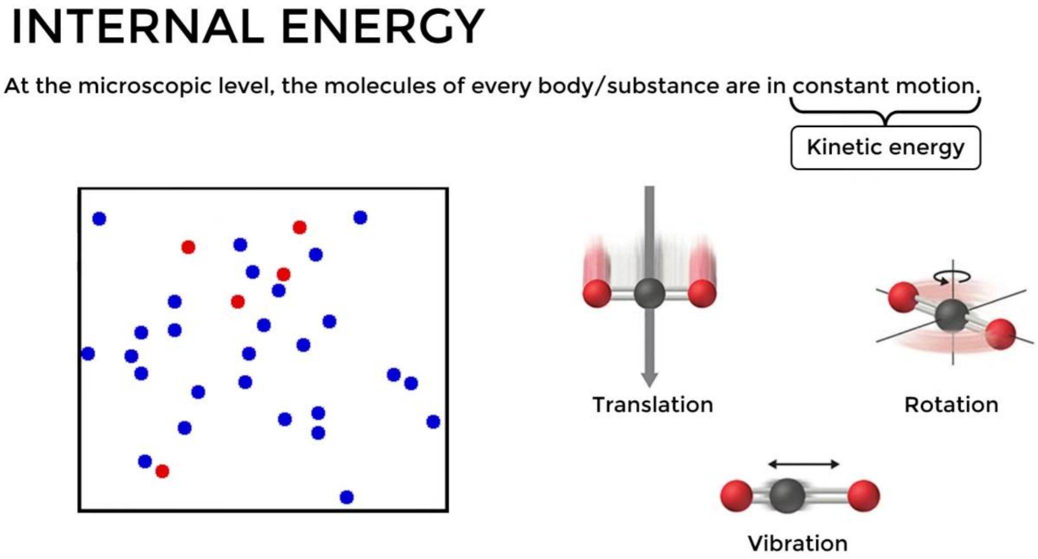
- Answers:

a.	Position	[m]	vector	$\vec{r}$
b.	Distance	[m]	scalar	
c.	Torque	[N/m]	vector	$\vec{T}$
d.	Force	[N]	vector	$\vec{F} = m\,\vec{a}$
e.	Energy	[J]	scalar	
f.	Power	[J/s = W]	scalar	
g.	Momentum	[kg·m/s]	vector	$\vec{p} = m\,\vec{v}$
h.	Speed	[km/h]	scalar	
i.	Velocity	[km/h]	vector	$\vec{v}$
j.	Mass	[kg]	scalar	Mass is a scalar quantity
k.	Weight	[N]	vector	Weight is a force (a vector) that can change based on the gravitational field strength, and is directed toward the center of the Earth

Speed vs. *Velocity* difference

Temperature vs Heat

- Difference Between Temperature and Heat
  - Aviation Theory: www.youtube.com/watch?v=sE98Zt1wtLc [18:18]



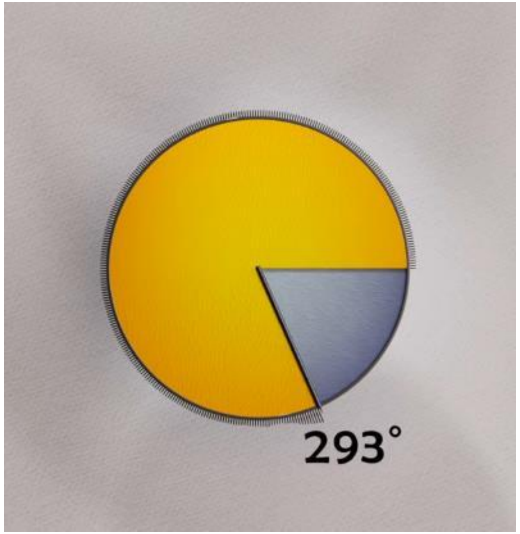
Angular notions

- Angular units of measurement
- Solid angle
- Some pitfalls to avoid

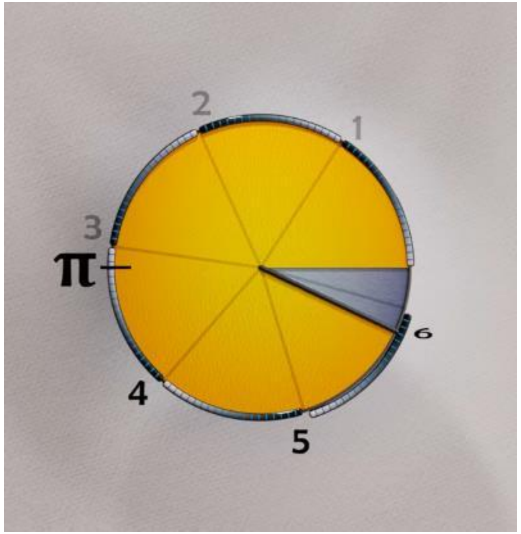
Degrees and radians

- The origin of Degrees and Radians
  - www.youtube.com/watch?v=glmeLc3dZf4

360° in a circle

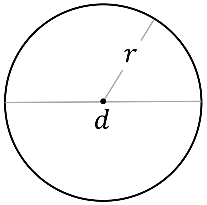


$\mathbf{2\pi}$ **sr** over the circumference



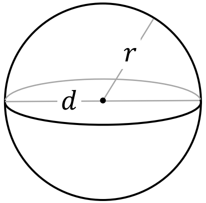
Basic relations

Area of a circle: $A = \pi\, r^2$

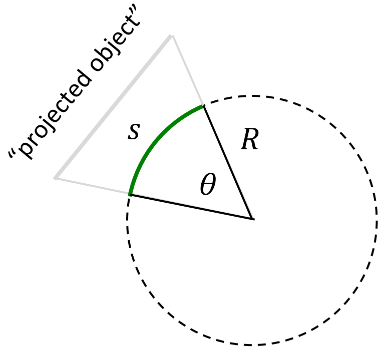


Surface of a sphere: $S = 4\pi\, r^2$

Volume of a sphere: $V = \frac{4}{3}\pi\, r^3$



- **Planar angle** [° or rad]
  - Length of a circle segment, s, bounded by the angle θ, divided by the radius of the circle, R

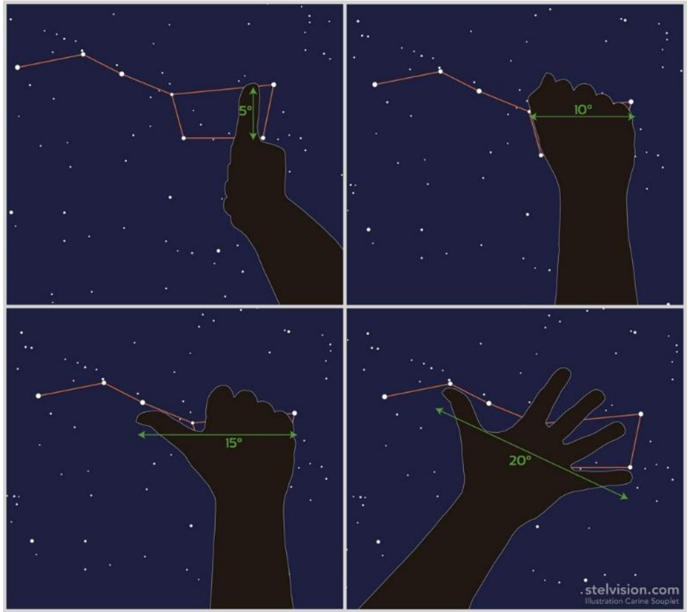


s = Arc length

$$\theta = \frac{s}{R}$$

Angular notions – Angular units of measurement ° ′ ″

- **DEGREE °**
  - Diameter of our little finger at arm's length ≈ **1°**
  - Andromeda Galaxy = 3° × 1°
  - Angular diameter of the **Moon ≈ 1/2° ≈ Sun**
    → allowing Total Solar Eclipse!



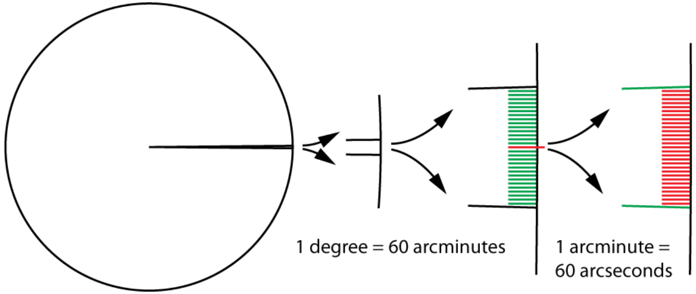
- **ARCMINUTE ′** minute of arc: 1**′ =** 1/60°
  - **1′ ≡ ladybug** (3 mm) **at 10 m**
  - 10 cents à 62 m; 25 cents à 82 m; 2$ à 100 m;
  - Soccer ball at 800 m; garbage can lid at 2 km
  - Angular diameter of **Venus = 1′..1.6′**

  → Angular diameter of the Moon = 31′

  Resolution limit of the human eye (20/20)

Angular notions – Angular units of measurement ° ′ ″

- **ARCSECOND ″** or second of arc: 1″ = 1/60′ = 1/3600°
  - **1″ ≈ human hair diameter** (50 μm) **at 10 m**
  - Resolution of the best binoculars; Galileo telescope
  - Moon's crater (1.5 km); etc.
  - Angular diameter of Jupiter = 30″..50″
  - Mars = 4″..25″ ; Neptune = 2.3″ ; Pluto = 0.1″
  - ρ Cassiopeia = 0.0072″ ; Proxima Centauri = 0.001″



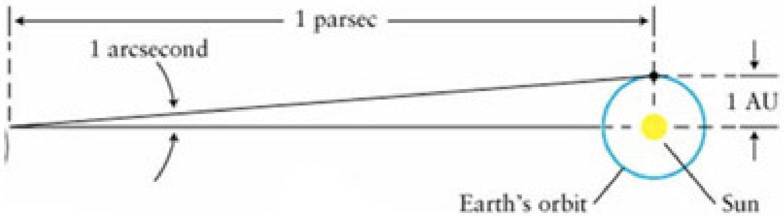
- The unit of arc second itself was first developed in 1670 by astronomer Jean Picard in an attempt to measure the size of the Earth more accurately. Since then, arc seconds have become an essential tool for astronomers, allowing them to measure the ever-smaller details of the night sky
- Since the development of the arc second in the 17th century, it has become a cornerstone of modern astronomy. Today, arc seconds are used as a unit of angular measurement by astronomers globally, allowing them to accurately measure objects in the sky that are too small to be measured using other units

  https://optcorp.com/blogs/customer-highlights/what-is-an-arc-second-in-astronomy

Parsec

- The trigonometric parallax of an object at a distance of 1 parsec (pc) is 1 arcsecond

1″ ≡ 1 AU at a distance of 1 parsec ← definition




```
arcsec = 1/60^2;  % [°]

% Definition: tan(arcsec) = 1 AU / 1 parsec
parsec = Earth.AU / tand(arcsec);  % = 3.086e13 [km]

% Number of light years in 1 parsec
ly_in_parsec = parsec / LightYear_km;  % [parsec/ly] ? 3.26156
```

- Turbulence in Earth's atmosphere causes point sources, such as stars, to be smeared out and twinkle. The extent of this smearing out is referred as "seeing". Under the best atmospheric conditions, point sources will be spread out to an angular diameter of about 0.5″ (½ arcsecond).
  - There are no stars beyond the Solar System with parallaxes equal to or greater than 1 arcsecond. The nearest star system to the Sun, that of Alpha Centauri, lies about 1.3 parsecs away and has a parallax of 0.75″.

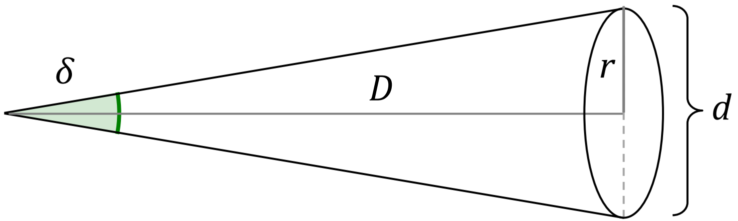
Angular diameter

- 360° = 2π radians; complete circle

  d is the object diameter ($\equiv 2\,r$), D is distance → δ is angular diameter

Definition:

$$\boxed{\delta \equiv 2\arctan\left(\frac{r}{D}\right)}$$

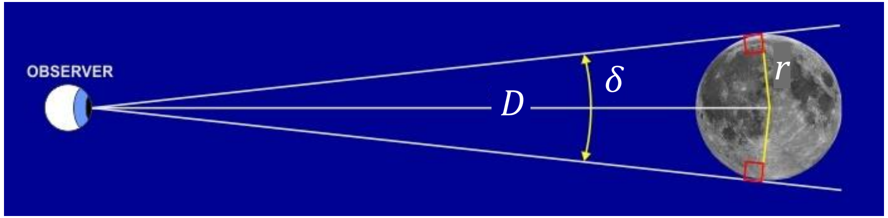


Approximations:

$$\delta \approx \arctan\left(\frac{d}{D}\right) \quad d \ll D$$

$$\lim_{x\to 0} \operatorname{atan}(x) = x$$

$$\delta \approx \mathrm{DEG}\left(\frac{d}{D}\right) \quad d \ll D$$



← notice the right angle

Exercise: Angular diameter

- Calculate the size of a drawn point on the classroom wall located 10 m away corresponding to: **1°**, **1′**, **1″**

Answer:

```
D = 10.0;  % [m] observation distance
angles = [1, 1/60, 1/60^2];  % [°] (full angle)
r = D * tand(angles / 2)  % [m] ← according to definition
d = 2 * r  % [m] diameter from radius
d .* [100, 1000, 1e6] → ø = 17 cm, 3 mm, 48 µm
```

Some analysis

```
%% SOL_Angular_diameter
r = Moon.radius_km  % = 1737.4 km
d = 2 * r
D = Moon.a % = 384399 km

% Angular diameter (Full angle)
delta = 2 * atand(r/D)  % 0.517925°

% approximation
delta = atand(d/D)      % 0.517915°
  delta = atand(d/D / (1 - (r/D)^2)) % ← to be exact

% approximation: atan(x) ≈ x for x << 1
delta_rad = 2 * r/D     % 0.00904 rad
delta = 2 * DEG(r/D)    % 0.517929°

Moon.Angular_diameter   % 0.517925°

%---- test for large angle → approximation does not hold
r = 1
D = 5
res = [ 2 * atand(r/D);  % 22.620°
          atand(2*r/D);  % 21.801° (3.6% off)
        2 * DEG(r/D) ]   % 22.918° (1.3% off)
pc_err = 100 * (res(2:3) / res(1) - 1)
```

→ start by initializing useful variables:

```
[Earth, Moon, Sun] = AstroConstants();
```

→ ask ChatGPT:

"why, when computing angular diameters, 2 atan(r/D) is not equivalent to atan((2r)/D)?"

Exercise: Angular diameter (continued)

- How far away should a tennis ball (ø = 6.7 cm) be placed so that it appears to have the same angular size as the planet Neptune?
- What is its angular diameter as seen by an observer on Earth? How does its size compare to that of the Moon?

```
NAIF_ID = 899;  % Neptune #
k = find(Planets.NAIF_ID == NAIF_ID);  % (index in table)
r_Neptune = Planets.R(k);  % [km] radius = 24766 km
D_Neptune = Planets.a(k) * Earth.AU;  % [km] = 4.50411e9 km

r_ball = (6.7/2) / 100;  % [m] radius from diameter

% r_ball / x == r_Neptune / D_Neptune
D_ball = r_ball / (r_Neptune / D_Neptune)  % → 6092 km
D_ball / (2 * Earth.r1_km)  % ≈ 0.955 ≈ 1 Earth radius

theta_Neptune = atan(2 * r_Neptune / D_Neptune)  % [rad] ≈ 2 * r_Neptune / D_Neptune
theta_Neptune / 1e-6  % → 11 mrad
rad2arcsec(2 * r_Neptune / D_Neptune)  % → 2.3"
  DEG(2 * r_ball / D_ball) * 60^2  % (idem)
Moon.Angular_diameter / DEG(theta_Neptune)  % ≈ 822 times smaller than the Moon
```

$2\,\mathrm{atan}(r/D) \neq \mathrm{atan}(2r/D)$

- ChatGPT: "why, when computing angular diameters, 2 atan(r/D) is not equivalent to atan((2r)/D)?"

Because the arctangent function is **non-linear**, so doubling its argument is **not** the same as doubling its value.

For angular diameter, the correct geometry gives the full angle as

$$\alpha = 2\arctan\left(\frac{r}{D}\right)$$

where r is radius and D is distance.
If you try to replace that with

$$\arctan\left(\frac{2r}{D}\right)$$

you are implicitly assuming that

$$2\arctan(x) = \arctan(2x)$$

which is false except in a limiting case.

Mathematically, the tangent **double-angle identity** shows why:

$$\tan(2\theta) = \frac{2\tan\theta}{1-\tan^2\theta}$$

Let $x = \tan\theta$. Then

$$2\theta = \arctan\left(\frac{2x}{1-x^2}\right)$$

not $\arctan(2x)$. So in terms of arctan:

$$2\arctan(x) = \arctan\left(\frac{2x}{1-x^2}\right)$$

Therefore

$$2\arctan\left(\frac{r}{D}\right) \neq \arctan\left(\frac{2r}{D}\right)$$

unless $x = r/D$ is extremely small so that x^2 is negligible. That is why in astronomy, for small angular sizes, we often approximate:

$$\boxed{2\arctan\left(\frac{r}{D}\right) \approx \frac{2r}{D}}$$

but this is an approximation, not an identity.

Some analysis (continued)

```
%---- test for Sun
r = Sun.radius_km % = 695700 km
d = 2 * r
D = Earth.AU % = 149597870.7 km
delta = 2 * atand(r/D)   % 0.53295° == Sun.Angular_diameter

% δ_Moon < δ_Sun? why? → This would mean no total solar eclipse is possible…
delta_Moon_lims = 2 * atand(Moon.radius_km ./ [Moon.Apogee_km, Moon.Perigee_km])
% = 0.491° .. 0.548°
```

→ The Moon's angular diameter varies from its perigee (closest position to Earth, where the Moon appears the largest) to its apogee (farthest position to Earth, where the Moon has its smallest angular diameter).

Total solar eclipses occur under the condition:

$$\delta_{☾}(\nu) > \delta_{\odot}$$

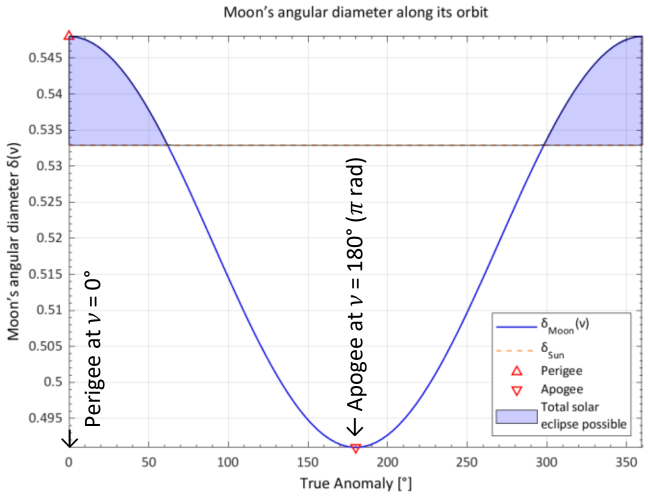
Concepts introduced later on the course

$$\frac{2\, r_{☾}}{R(\nu)} > \delta_{\odot}$$

← $R(\nu)$ is the Moon's orbit radius in function of True Anomaly ν

$$R(\nu) = \frac{a(1-e^2)}{1+e\cos(\nu)}$$

← a & e are Moon's orbit semimajor axis and eccentricity

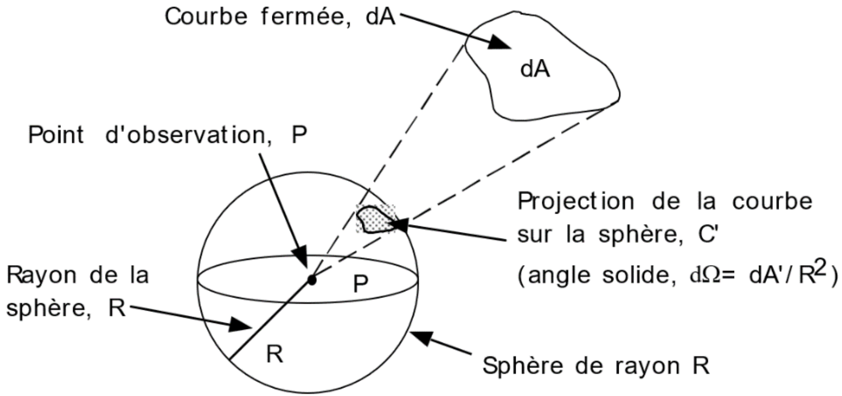


(back to Introduction)

Solid angle

- A solid angle is the three-dimensional analog of a plane angle. It quantifies how large an object appears to an observer by measuring the portion of space it subtends.
- A solid angle Ω is defined as the area A that a surface cuts out on a unit sphere, divided by the square of the sphere's radius.

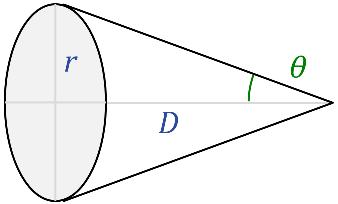
$$\boxed{\Omega = \frac{A}{r^2}}$$



- A **steradian** [sr] is the SI unit of solid angle
  - The entire sphere has: $\Omega_{\text{full sphere}} = 4\pi$ sr
- The square degree [°²] is a non-SI unit for measuring solid angles.
  - The conversion factor between steradians and square degrees is $(180/\pi)^2$ square degrees per steradian.
- Celestial Sphere surface
  - The surface of the celestial sphere, the full sphere centered on the Earth, measures ≈ 41,253 square degrees [deg²]

Solid angle

- Cone with angular opening θ (half angle)



$$\theta = \text{asin}(r/D)$$ ← angle to the tangent side

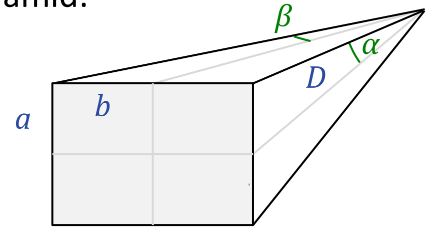
$$\Omega = 2\pi\,(1 - \cos(\theta))$$ ← Definition

$$\Omega = 4\pi\,\sin^2(\theta/2)$$

$$\boxed{\Omega = 2\pi\left(1 - \sqrt{1 - (r/D)^2}\right)}$$

$$\Omega \approx 2\pi\left(\frac{r}{D}\right)^2 \quad r \ll D$$ ← small angle approximation

- Square or rectangular pyramid:



$$\Omega = 4\,\text{asin}\left(\frac{l\,b}{\sqrt{(l^2 + 4D^2)(b^2 + 4D^2)}}\right)$$

$$\Omega = 4\,\text{asin}\big(\sin(\alpha)\sin(\beta)\big)$$

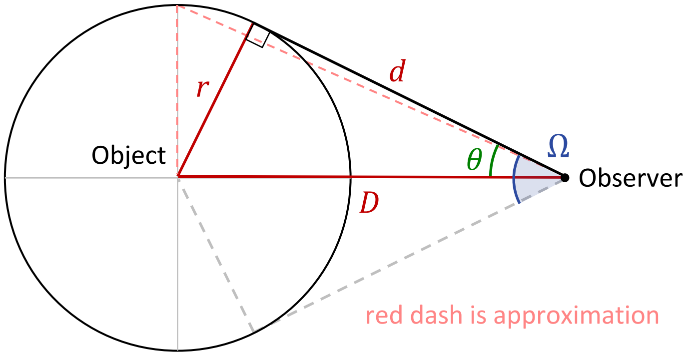
Solid angle computation

$\Omega = 2\pi(1 - \cos\theta)$ ← definition

θ = half the angular diameter of the object, as seen from the observer

r = object radius
(e.g. radius of the Earth $R_\oplus$)

D = distance to *center* of object
(e.g. $R_\oplus + h$ altitude of satellite)



The solid angle is defined from the perspective of the observer. It measures how much of the observer's field of view is occupied by an object.

$$r^2 + d^2 = D^2$$

$$\cos\theta = \frac{d}{D} = \frac{\sqrt{D^2 - r^2}}{D} \quad \text{[rad]}$$

$$\boxed{\Omega = 2\pi\left(1 - \sqrt{1 - (r/D)^2}\right)}$$

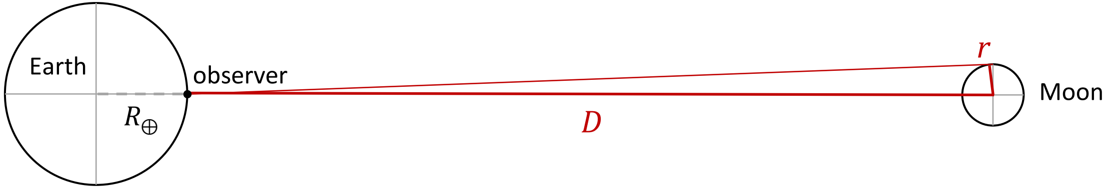
$\Omega \approx \pi\,(r/D)^2$ when $\frac{r}{D} < 10^{-4}$

Observer-centric calculation

When computing solid angle as seen by an observer on Earth's surface, use:

$$D = REM - R_{\oplus} \cos(\phi)$$

R_{EM} : center-to-center Earth-Moon distance (≈384400 km ≡ `Moon.a`)
$R_{\oplus}$: Earth's mean radius (≈6371 km ≡ `Earth.radius_km`)
ϕ: zenith angle of the Moon (0 if directly overhead)



→ $\Omega_{☾}$ = $6.636{\times}10^{-5}$ sr (3.4% larger than the value $6.418{\times}10^{-5}$ sr for an observer at center of the Earth

- Since solid angle is fundamentally about what the observer sees, and astronomical observations are made from Earth's surface, the surface-to-Moon distance is the physically meaningful quantity to use
- For the most accurate calculation, the Moon's elliptical orbit and the observer's specific latitude/longitude must be accounted for, but the key point is using the observer's position rather than Earth's center as the reference point.
- For radiance or irradiance observed from Earth, always use the observer-to-Moon distance for best accuracy

Solid Angle Ω

```
function Omega = SolidAngle(r, D, type)
% Computes the spherical angle Ω [sr] of a sphere of radius r seen at a distance D (from observer to sphere center)

    if type == "circular"

        % Ω = 2π (1 – cos(θ))
        if r / D > 1e-4 % Formula for large angles
            Omega = 2*pi * (1 - sqrt(1 - (r./D).^2));
        else % approximation for small angles
            Omega = pi * (r./D).^2;
        end

    else % square

        Omega = 4 * asin(r^2 / (r^2 + 4 * D^2));  % [sr]

    end

end % function
```

$\Omega_{Sun} \equiv 6.794\times10^{-5}$ sr

```
r = Sun.radius_km;
D = Earth.AU – Earth.radius_km;
Omega_Sun = SolidAngle(r, D)
```

$\Omega_{Moon} \equiv 6.636\times10^{-5}$ sr

```
r = Moon.radius_km;
D = Moon.a – Earth.radius_km;
Omega_Moon = SolidAngle(r, D)
```

Because AU & *a* are defined from the body center

$\Omega_{LEO} \equiv 3.931$ sr

```
r = Earth.r1_km;
D = 500 + Earth.r1_km;
Omega_LEO = SolidAngle(r, D)
```

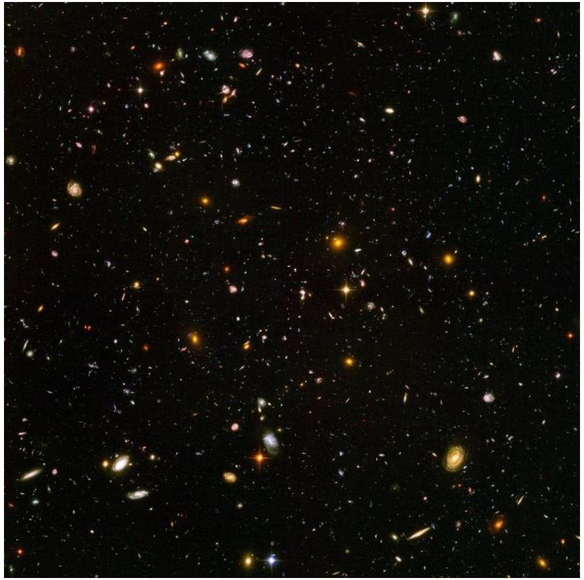
Altitude defined *above* body surface

(see illustration in Solid angle computation)

Hubble Ultra-Deep Field

- The Hubble Ultra-Deep Field (HUDF) is a deep-field image of a small region of space in the constellation Fornax, containing an estimated **10,000 galaxies**. The original data for the image was collected by the Hubble Space Telescope from September 2003 to January 2004. It includes light from galaxies that existed about 13 billion years ago, some 400 to 800 million years after the Big Bang
- The HUDF image was taken in a section of the sky with a low density of bright stars in the near-field, allowing much better viewing of dimmer, more distant objects. Located southwest of Orion in the southern-hemisphere constellation Fornax
- The rectangular image is **FOV = 2.4′ × 2.4′** to an edge (Field of View)
- This is about one-tenth of the angular diameter of a full Moon viewed from Earth, smaller than a 1 mm² piece of paper held 1 m away, and equal to roughly 1/26 M of the total area of the sky

  https://en.wikipedia.org/wiki/Hubble_Ultra-Deep_Field
  https://esahubble.org/images/heic0611b/



Observation time = 1 M s, 400 orbits, 11 days

Exercise: Solid Angle

- Compute the fraction of the sky corresponding to the HUDF FOV

```
Answer:

CS_sr = 4*pi;  % [rad]
CS_deg2 = CS_sr * DEG(1)^2;  % = 41252.96°²

ratio = CS_deg2 / FOV_deg2;
ratio / 1e6  % → 1/25.8 M
```

- If 10,000 galaxies were observed in this area, assuming uniform distribution across the CS, how many galaxies are there in total?

```
Answer: ratio * 1e4 / 1e9  % ≈ 2.6×10¹¹ = 258 G (billion)
```

- What is the current estimate of this number?
  - *Earlier* estimations (pre-Hubble deep surveys and models) placed the number in the range of 100–200 billion galaxies, but improved models indicate this was a significant undercount of faint, small, or distant systems.
  - *Recent* studies based on deep-field surveys from the Hubble Space Telescope and advanced extrapolation models suggest the observable universe contains on the order of 2 trillion galaxies (**$\approx 2\times10^{12}$**).
    This figure accounts for many faint and distant galaxies that current telescopes cannot directly see.

Some pitfalls to avoid

- **Radians** [rad] vs **Degrees** [°] (default)
  - Be careful, different functions, toolboxes have different input & output arguments → *always check*!
  - `cos(in_rad)` ≡ `cosd(in_deg)` (in MATLAB)
  - `out_deg = DEG(in_rad)`; or `out_deg = 360 * in_rad / (2*pi)`
  - IMP: Kepler equation *requires* radians: $M = E - e\sin(E)$
  - The `unwrap()` function works in radians: example in `OrbitPropagation.m` f and g computation

- **Order in arguments**:
  - `[lat,lon] = scircle1(el,az), f_fix_lines(lat,lon), projfwd(lat,lon)`
    `vs. view(az,el), Sph2Cart(az,el), etc.`

  → always check which one is which!

- **Modular arithmetic**, be careful:
  - −17° ≡ 343° (= `mod(-17, 360)`)
  - e.g. Longitude: −180°..+180° vs 0..360°: → `mod(lat + 180, 360)-180`

Planar geometry

- Law of Cosines:

$$\boxed{\begin{aligned} a^2 &= b^2 + c^2 - 2\,b\,c\cos(A) \\ b^2 &= c^2 + a^2 - 2\,c\,a\cos(B) \\ c^2 &= a^2 + b^2 - 2\,a\,b\cos(C) \end{aligned}}$$

- Law of Sines

$$\boxed{\frac{a}{\sin(A)} = \frac{b}{\sin(B)} = \frac{c}{\sin(C)}}$$

- Area & heights

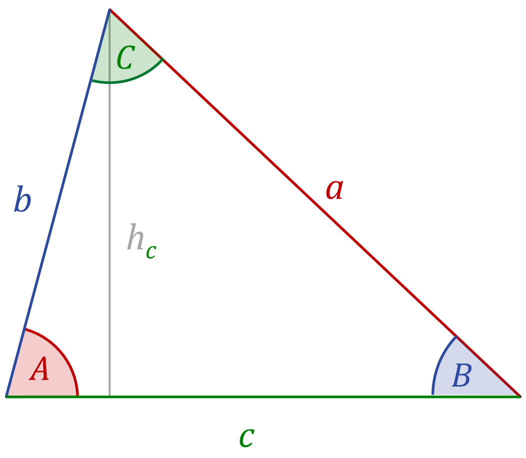
$p = a + b + c\,;\, s = p/2$ perimeter & semi-perimeter

$T = \sqrt{s(s-a)(s-b)(s-c)}$ Area

$h_a = b\sin(C) = 2\,T/a$ perpendicular to a

$h_b = c\sin(A) = 2\,T/b$ perpendicular to b

$h_c = a\sin(B) = 2\,T/c$ perpendicular to c



Orbital geometry

Trigonometric functions

Useful trigonometric equivalences

- Useful trigonometric equivalences

$$\operatorname{asin}(x) \equiv \operatorname{acos}\left(\sqrt{1 - x^2}\right)$$

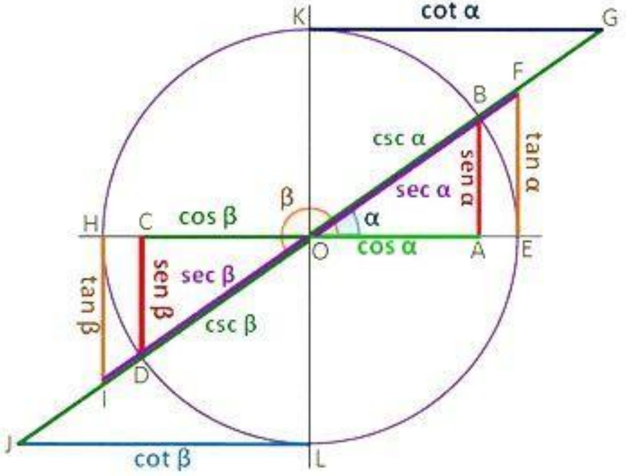
other useful…

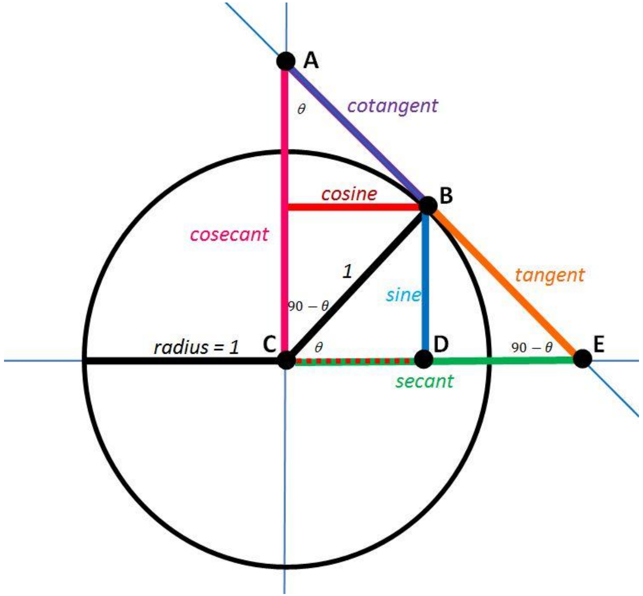
For small angles (`SolidAngle.m`)

$$1 - \sqrt{1 - x^2} \approx \frac{x^2}{2} \quad \text{for } x \ll 1$$

$$\begin{aligned} \sin(\alpha \pm \beta) &= \sin\alpha\cos\beta \pm \cos\alpha\sin\beta \\ \cos(\alpha \pm \beta) &= \cos\alpha\cos\beta \mp \sin\alpha\sin\beta \end{aligned}$$

Trigonometric functions

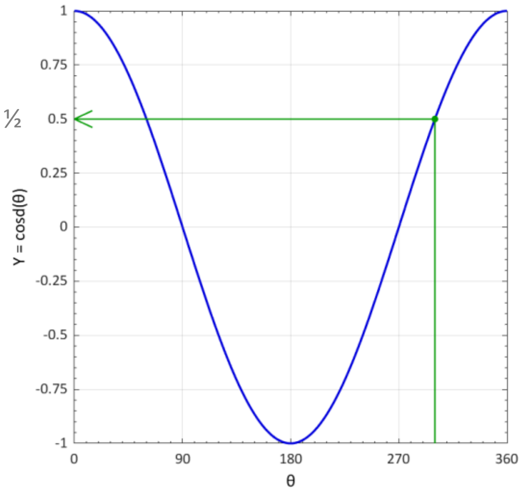




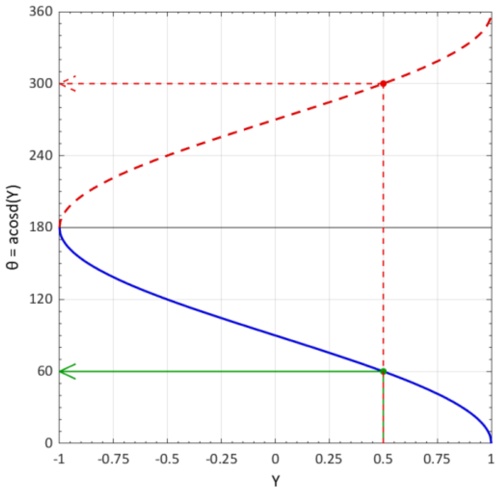
Inverse trigonometric functions: check quadrants

- **acos()** and **asin()**

cos(300°) = ½



⟷



acos & asin ambiguity: 2 *solutions*

$$\boxed{\begin{aligned}\cos(\theta) &\equiv \cos(360-\theta)\\ \sin(\theta) &\equiv \sin(\theta-360)\end{aligned}}$$

acosd(0.5) = 300°
also = 360 – 300 = 60°

→ Need to find right case with argument check

Example

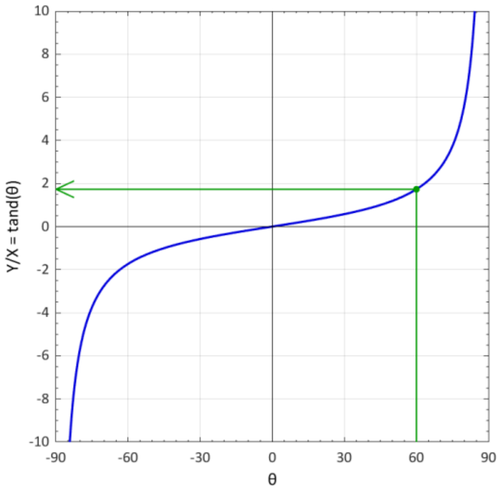
```
b = -20;
x = 8;
y = sind(b * x);

% find x:
x1 = asind(y) / b  % = 1

% → this is the solution we're looking for:
x2 = (-asind(y) - 180) / b % = 8
% or:
x3 = x1 - 360 / b % = 19


sind(b * x1) == sind(b * x2) == sind(b * x3)

b * (x1 + x2) == -180
mod(b * (x1 - x3), 360) == 0
```

→ See SOL Part II: Case of flight path azimuth angle $\beta = \mathrm{asin}(\cos(i)/\cos(\phi))$ used for ground velocity calculation

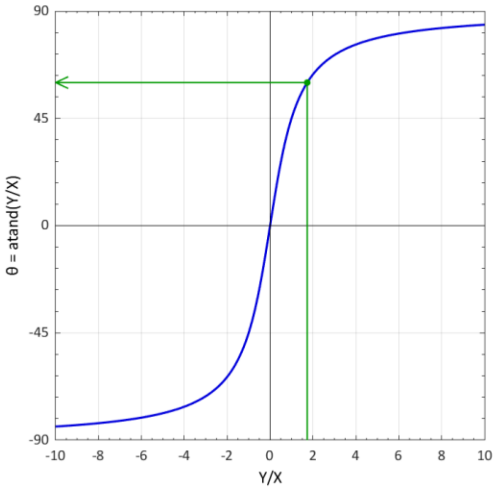
Inverse trigonometric functions: check quadrants

- **`tan()`** and **`atan()`**

tand(60°) = $\sqrt{3}$



$\longleftrightarrow$



no ambiguity with arc tangent

→ If possible, preferable to use formulas with `atan()` rather than `acos()` or `asin()` to avoid quadrant ambiguity

atan(y/x) vs atan2(y, x)

- The **`atan2()`** function is used to calculate the arctangent of y/x in the four quadrants, taking into account the signs of x and y to place the vector in the correct quadrant
  - For example, if $x = 1$ and $y = 1$, the standard inverse tangent returns 45° from the x-axis.

    `atand(1/1)` == +45° ← OK

  - However, if $x = -1$ and $y = -1$, the standard inverse tangent also returns 45° even though the vector is in the third quadrant and the correct value is −135°.

    `atand(-1/-1)` == +45 ← NO (signs information is lost through division)
    `atan2d(-1, -1)` == -135° ← OK

- The **`atan2()`** function returns values in the closed interval $[-\pi, \pi]$
- In contrast, the **`atan()`** function returns results that are limited to the interval $[-\pi/2, \pi/2]$

  → *When possible*, preferable to use formulas with **`atan2()`** rather than **`atan()`** to avoid quadrant ambiguity

  Counter example: Direct analytical solution for day-night separator $\varphi = f(\lambda, \delta)$

```
✗ phi = atan2d(-cosd(lambda - Sun_info.Lon), tand(delta)); ← No for delta < 0
✓ phi = atand(-cosd(lambda - Sun_info.Lon) ./ tand(delta));
```

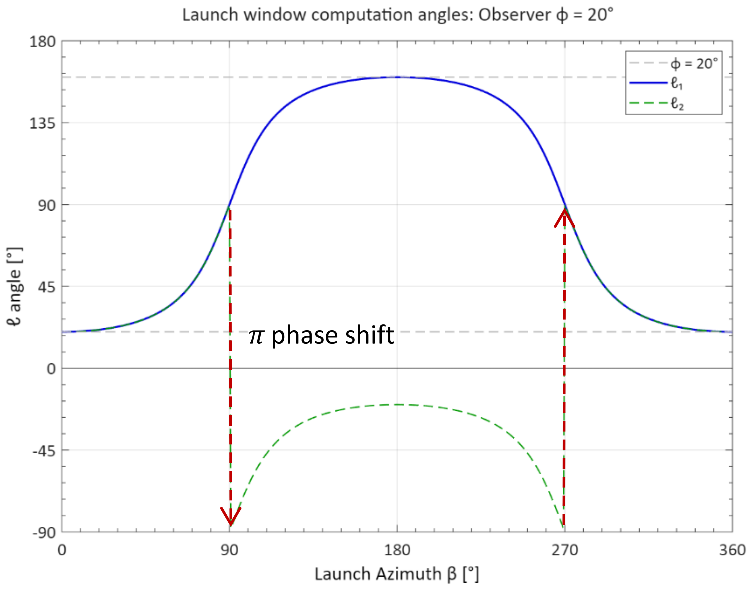
→ Note that Excel uses **`atan2(x, y)`** with *inversed arguments*

atan(y/x) vs atan2(y, x)

φ = constant

$\ell_1 = \mathrm{atan2}(\tan(\varphi), \cos(\beta))$ $[\pm 180°]$ ← better

$\ell_2 = \mathrm{atan}(\tan(\varphi)/\cos(\beta))$ $[\pm 90°]$

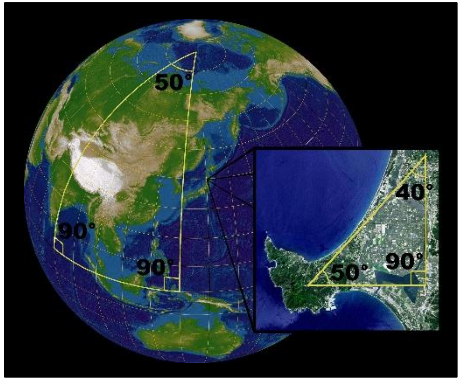


IMP: Visual study helps to better understand formulas

→ be sure to properly understand the behavior in different quadrants, in order to *use the appropriate functions* or *apply manual quadrant correction* to ensure getting correct results

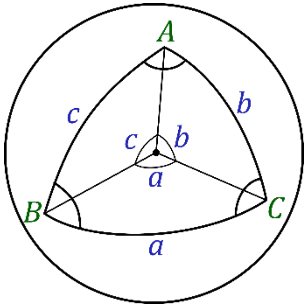
Spherical trigonometry

- Spherical trigonometry is the branch of spherical geometry that deals with the metrical relationships between the sides and angles of spherical triangles, traditionally expressed using trigonometric functions
- Analogous to Euclidean plane trigonometry, but with some important differences
- *Spherical trigonometry is of great importance for calculations in astronomy, geodesy, and navigation*



The sum of the angles of a spherical triangle ≠ 180°.
A sphere is a curved surface, but locally the laws of the flat (planar) Euclidean geometry are good approximations.

In a small triangle on the face of the Earth, the sum of the angles is only slightly more than 180°.

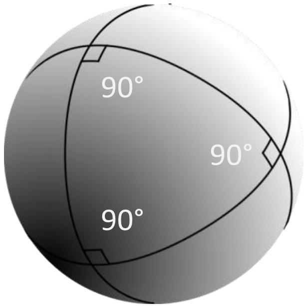


Spherical law of cosines:

$$\boxed{\begin{aligned}\cos(a) &= \cos(b)\cos(c) + \sin(b)\sin(c)\cos(A)\\ \cos(b) &= \cos(c)\cos(a) + \sin(c)\sin(a)\cos(B)\\ \cos(c) &= \cos(a)\cos(b) + \sin(a)\sin(b)\cos(C)\end{aligned}}$$

As a special case, for $A = 90°$, one obtains the spherical analogue of the Pythagorean theorem:

$$\cos(a) = \cos(b)\cos(c)$$



On the sphere, geodesics are Great Circles

Exercise

- Convert 12° 13′ 14″ into decimal degrees
- Convert 12.1314 into DMS
- Convert 12h 13m 14s into decimal degrees
- 45° to radians
  - `45/360 * 2*pi` → RAD(45)
- 0.123 rad to degrees
  - 0.123/(2*pi) * 360 → DEG(0.123)

Vectors

- Dot Product, Inner, or Scalar Product
- Cross, Outer, or Vector Product

Vectorial notation

- Standard mathematical notation:
  - (Vectors in bold)

$$\boldsymbol{r} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}$$

$$r = |\boldsymbol{r}| \quad \leftarrow \text{magnitude of vector}$$

- Alternative mathematical notation:
  - Used in this presentation

$$\vec{r} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}$$

$$r = |\vec{r}|$$

- Programming code:
  - Upper/lowercase used in Matlab programs

```
R = [r_x; r_y; r_z];  % 3×n array
r = norm(R);  % Magnitude of 3D vector
```

Coordinates handling in Matlab

- **3 × *n*** arrays (3 for x,y,z)

```
xyz = Sph2Cart(0:90:360, 0:22.5:90, Earth.r1_km)
   size(xyz) ≡ [3 5]  % == [3×n]
Orbit.ECI
   pos: [3×201 double]
   vel: [3×201 double]
v = [1; 2; 3];  % == [1, 2, 3]'
```

- Allows easy rotation matrix multiplication

```
RotX(45) * xyz
[3 × n] = [3 × 3] * [3 × n]
```

- Use

```
plot3(xyz(1,:), xyz(2,:), xyz(3,:));  % for plot3(x, y, z)
→ Plot3(xyz); % ← simplified writing
```

- `[x, y, z] = Sph2Cart(ax, el)` and also `xyz = Sph2Cart(ax, el)`

Vectorial multiplication

1. Dot Product, Inner, or Scalar Product

$$\vec{R}_1 \cdot \vec{R}_2 = \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} \cdot \begin{bmatrix} x_2 \\ y_2 \\ z_2 \end{bmatrix} = x_1 x_2 + y_1 y_2 + z_1\, z_2$$

$$= r_1 r_2 \cos(\alpha)$$

- The inner product of two vectors produces a scalar that is equal to the product of the length of individual vectors and the cosine of the angle between them.
- Angle between two vectors:

```
alpha = acosd(dot(R1, R2) / (norm(R1) * norm(R2))
alpha = Angle(R1, R2)
```

- Eccentricity vector $\vec{e}$ pointing to perigee along the Apse line

```
Ev = 1 / Orbit.mu * ((v^2 - Orbit.mu / r) * R - dot(R,V) * V);
```

↑ vectorial products ↑

Vectorial multiplication

2. Cross, Outer, or Vector Product

$$\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 = \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} \times \begin{bmatrix} x_2 \\ y_2 \\ z_2 \end{bmatrix} = \begin{bmatrix} y_1 z_2 - y_2 z_1 \\ x_2 z_1 - x_1 z_2 \\ x_1 y_2 - x_2 y_1 \end{bmatrix}$$

$$= (r_1 r_2 \sin\alpha)\,\hat{u}_{r_3} = r_3 \hat{u}_{r_3}$$

$$\hat{u}_{r_3} = \hat{u}_{r_1} \times \hat{u}_{r_2}$$

- The outer product of two vectors $\overrightarrow{r_1}$ and $\overrightarrow{r_2}$ produces another vector $\overrightarrow{r_3}$ that is perpendicular to the plane of $\overrightarrow{r_1}$, $\overrightarrow{r_2}$ such that the cycle $\overrightarrow{r_1} \to \overrightarrow{r_2} \to \overrightarrow{r_3}$ makes a right-handed triad. The length of $\overrightarrow{r_3}$ is equal to the product of the length of individual vectors multiplied by the sine of the angle between them. Hence $\overrightarrow{r_3}$ is numerically equal to the area of the parallelogram made up of $\overrightarrow{r_1}$ and $\overrightarrow{r_2}$.
- The outer product is defined and applied only in 3D space.

```
H = [R(2)*V(3)-R(3)*V(2); R(3)*V(1)-R(1)*V(3); R(1)*V(2)-R(2)*V(1)];
H = cross(R0, V0);  % [km²/s] Angular Momentum vector
```

Rotation matrices

- 3 x 3
- 6 x 6

Rotation matrices

- Spherical astronomy sections in older textbooks rely heavily on the reader's knowledge of spherical trigonometry. For more complicated problems than a simple rotation, this technique becomes laborious.
  → Vector and matrix techniques come to rescue
- Many of the transformations are defined in terms of rotation matrices
- A rotation matrix is a matrix whose multiplication with a vector rotates the vector while preserving its length.
- There is a special group 3 × 3 rotation matrices R where: $|R| = \pm 1$ and $R^{-1} = R^T$
  – For transformations between coordinate systems we only use matrices with $|R| = +1$
- A coordinate rotation is a rotation about a single coordinate axis.
- The three coordinate rotation matrices are:

$$R_x(\theta), R_y(\phi), R_z(\psi)$$

- Three coordinate rotations in sequence can describe any rotation. The result matrix is:

$$R_{ijk}(\phi, \theta, \psi) = R_i(\phi)\ R_j(\theta)\ R_k(\psi)$$

- The angles are called Euler Angles

Rotation matrices

- Rotation matrix around x, y, or z axis (3×3) $M' = R_M(\theta) * M$
  - here the $*$ symbol represents the matrix multiplication (not the convolution operator)

→ We use here the standard mathematical convention, “right-handed” (Right-Hand Rule RHR), i.e. *counter-clockwise*

- https://en.wikipedia.org/wiki/Rotation_matrix
- www.mathworks.com/help/phased/ref/rotx.html
- https://reference.wolfram.com/language/ref/RotationMatrix.html.en

like Battin 1999 (p.85), Chobotov 2002 (4.108), etc.

but *unlike* Bate 1971 (p.77 Eq.2.6-8), Vallado 2013 (Eq. 3-15) ,
Curtis 2020 (Eq. 4.32), Montenbruck 2000 (p.27), Taff 1985 (p.37)
Heafner 1999, and others who use the transpose of these matrices*

simply a matter of convention and use

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

→ Matrix multiplication is not commutative.
The order of multiplication is extremely important!

→ Rotations are evaluated from right to left.

SOL Part II – Right-hand system

Rotation matrices

- Matrix rotation must work in both directions. Using matrix operations, the reverse process would use the inverses of each matrix, in the opposite order. Usually, matrix inversion is mathematically intensive.
  → However, rotation matrices are special
- A matrix is called *orthogonal* if the rows and columns are scalar components of mutually independent unit vectors. Here, all rows are mutually independent, so the rotation matrix is orthogonal.
- The inverse of an orthogonal matrix is the *transpose*, and the transpose is calculated by simply switching the rows and columns. In terms of the individual rotations, the order and signs of each angle are reversed, and for the complete matrix, the rows and columns are exchanged
- For example, a rotation matrix has the property of: `inv(R) == R'`

  `RotX(30)' == inv(RotX(30)) == RotX(30)^-1 == RotX(-30)` ← also negative angle

- Usage: One can write in 3×n line matrix notation:

```
R = RotX(5)     norm(R)≡1
A = [1:4; 2:5; 3:6]
B = R * A
```

Combined rotations

```
% Coordinate transformation matrix: Perifocal → ECI
% rotation by ω° in perifocal plane, followed by i° over x axis, followed by Ω° over z axis
```

```
Orbit.RotPeri2ECI = RotZ(Orbit.RAAN) * RotX(Orbit.i) * RotZ(Orbit.w);
```

```
Orbit.ECI.pos = Orbit.RotPeri2ECI * [P; W; 0];
```

→ Easier and clearer than to write the open matrix, as often seen in many textbooks:

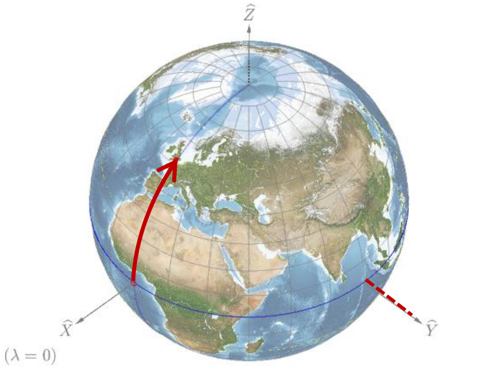
```
PQW_ECI = [ cosd(Omega) * cosd(w) - sind(Omega) * sind(w) * cosd(i),
           -cosd(Omega) * sind(w) - sind(Omega) * cosd(w) * cosd(i), ...
            sind(Omega) * sind(i);
            sind(Omega) * cosd(w) + cosd(Omega) * sind(w) * cosd(i), ...
           -sind(Omega) * sind(w) + cosd(Omega) * cosd(w) * cosd(i), ...
           -cosd(Omega) * sind(i);
            sind(w) * sind(i), cosd(w) * sind(i), cosd(i)];
```

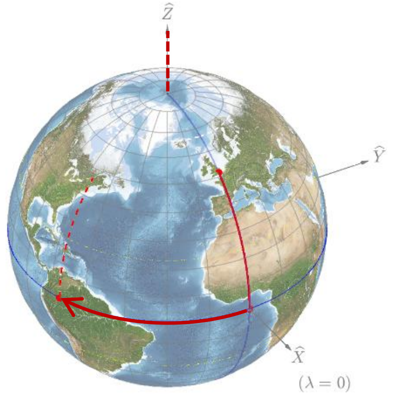
Here `cosd` and `sind` means function using an argument in *degrees* instead of *radians*

Rotations: sign

```
Earth = AstroConstants();

% ECEF (XYZ): Starting point at λ = 0°, ϕ = 0°
xyz = Sph2Cart(0, 0, Earth.r1_km);
Plot3(xyz, 'ro');

%--- Move up to latitude ϕ = +51.5° (Greenwich latitude)
Lat = +51.5; % [°N]
xyz_ = RotY(-Lat) * xyz;   % ← requires a negative sign, as latitudes are
                           %   measured clockwise, while the RotY rotation
                           %   operator is defined counter-clockwise

Plot3(xyz_, 'ro');

%--- Move westward to longitude λ = 71.2°W (Quebec City longitude)
Lon = -71.2; % [°W] negative sign for westward longitudes
xyz_ = RotZ(+Lon) * xyz;   % ← positive here, as longitude are measured
                           %   counter-clockwise, the same as RotZ operator

Plot3(xyz_, 'ro’);
```

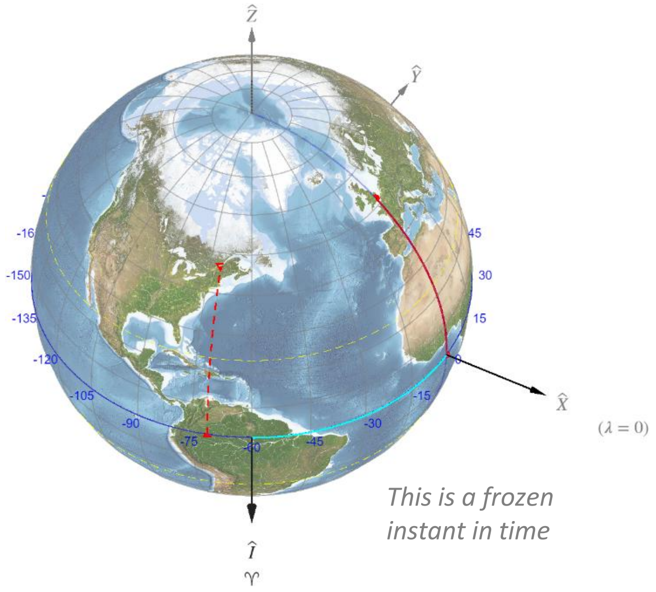
```
RotZ(lambda) * RotY(-phi) * [rho;0;0] == Sph2Cart(lambda, phi, rho)
```





Rotations between ECI and ECEF

```
% Greenwich Mean Standard Time (arbitrary here)
GMST = +60;  % [°] from 1stA ♈ (I) to λ=0° (X) (Greenwich)

% IMP → must rotate in the opposite direction (or subtract)
%       to convert ECI to ECEF

%--- Draws GMST angle
lambda = linspace(-GMST, 0);
arc = Sph2Cart(lambda, 0 * lambda, Earth.r1_km*1.002);
Plot3(arc, 'c-', 'LineWidth',1.5);

%--- Draws I axis in the ECEF frame
sf = 1.5 * Earth.r1_km;
X_arrow = RotZ(-GMST) * [0, sf; 0, 0; 0, 0];
mArrow3(X_arrow(:,1), X_arrow(:,2));

% → The PM position in the ECI frame would be computed with +GMST

view(-GMST + 90, 45);  % View from I (+90° is because graph 0° origin is in y axis)
```



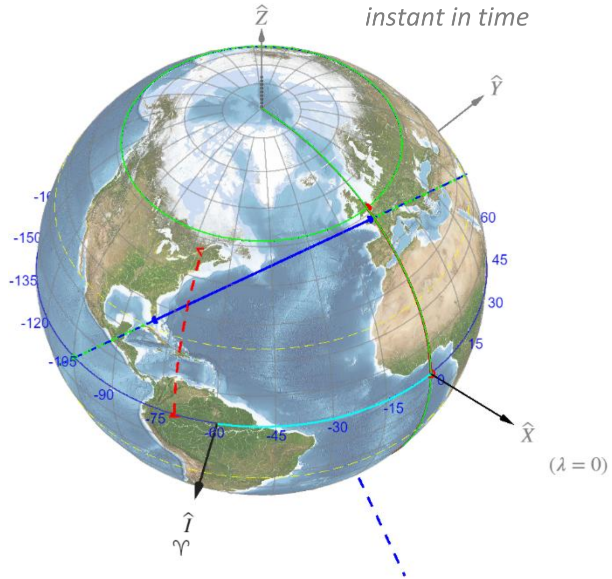
This is a frozen instant in time

SOL Part II – GMST

Great Circle

```
% Shortest path from two points along the globe

% 1) usig GreatCircle2 based on two coordinates on the globe (λ₁,ϕ₁) → (λ₂,ϕ₂)
GC = GreatCircle2(lat1, lon1, lat2, lon2); % between points
% computes Longitude at the node (α₀)
GC.Pos = GC.xyz * Earth.r1_km;
Plot3(GC.Pos, 'b-', 'Linewidth',2);

% 2) Using GreatCircle with main angles
[lat, lon, xyz] = GreatCircle(GC.lon0 + 90, -GC.alpha_0); % West Offset
xyz = xyz * Earth.r1_km;
Plot3(xyz, 'c:', 'Linewidth',2);

xyz = Sph2Cart(GC.lon0, 0, Earth.r1_km);
Plot3(xyz, 'c.', 'MarkerSize',20);
[lat, lon, xyz] = GreatCircle(90 + GC.lon0, 0);  % → Prime Meridian
xyz = xyz * Earth.r1_km;
Plot3(xyz, 'c--', 'Linewidth',1.5);
```

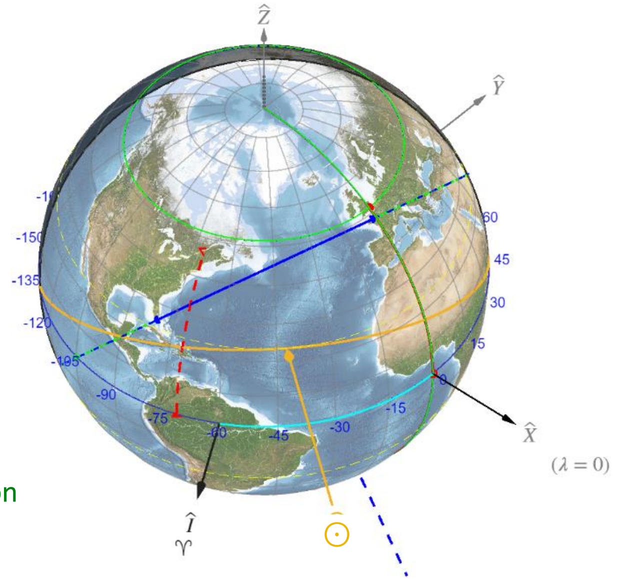
This is a frozen instant in time



GreatCircle → theory presenter later on

Sun ray and ecliptic

```
EQ = EquinoxSolstices(2024);
Date_UTC = EQ(2); % Summer Solstice: maximum Sun declination
                  % (on Tropic of Cancer)
Date_UTC = EQ(2) + 18;  % [hr] to be in daylight

GMST = ComputeGMST(Date_UTC);
Sun = SunEphemeris(Date_UTC);

DrawSun_and_Shadow(Sun.Longitude, Sun.Declination, Earth_sphere);

%--- Draws Ecliptic plane
theta = linspace(0, 360, 101);
ring = [cosd(theta); sind(theta); 0 * theta];
Obliquity = EarthObliquity();  % ϵ = 23.436° (2024)
S = Earth.r1_km * RotX(Obliquity) * ring;  % ← positive, as inclination
S = RotZ(-GMST) * S;  % ← negative here, as from ECI to ECEF
Plot3(S, 'y-');
```

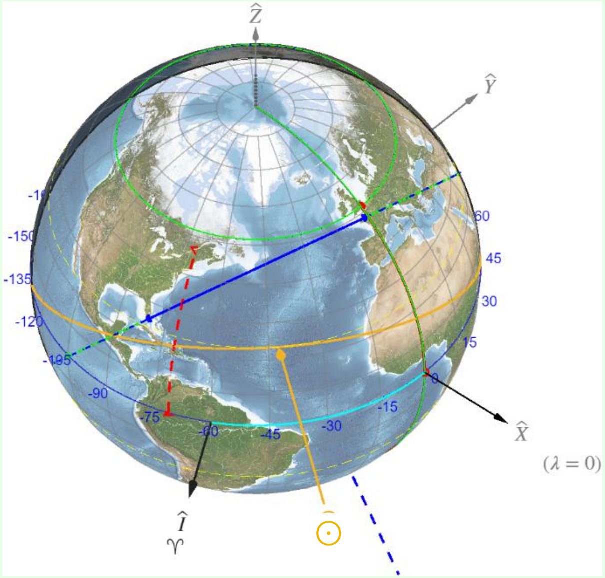


→ This exercise combines many concepts presented in Part 1 of the course

Perpendicular vector

E

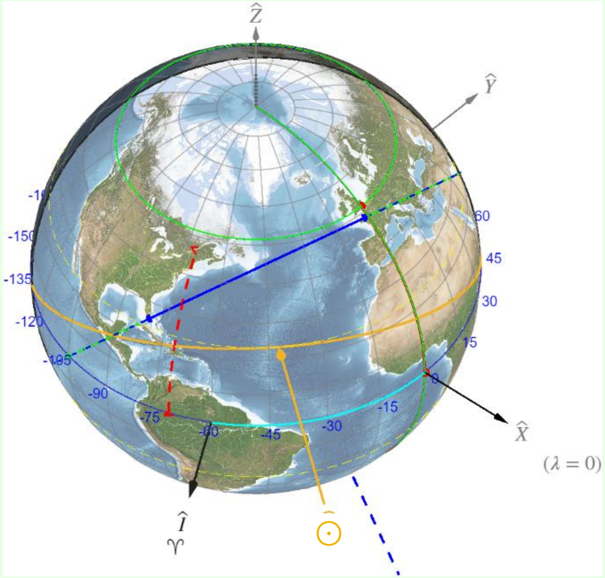
→ This exercise combines many concepts presented in Part 1 of the course



Perpendicular plane

E

→ This exercise combines many concepts presented in Part 1 of the course



IMP for 3D rotations

- Rotation in function of a line of sight

```
view_vector = [0.5; 0.5; 0.5];
[Az, El] = Cart2Sph(view_vector); % → Az = 45°, El = 35.26°
xyz_ = RotZ(Az) * RotY(90 - El) * xyz;   % ← In that order
```

- Rotation to bring a plane horizontal

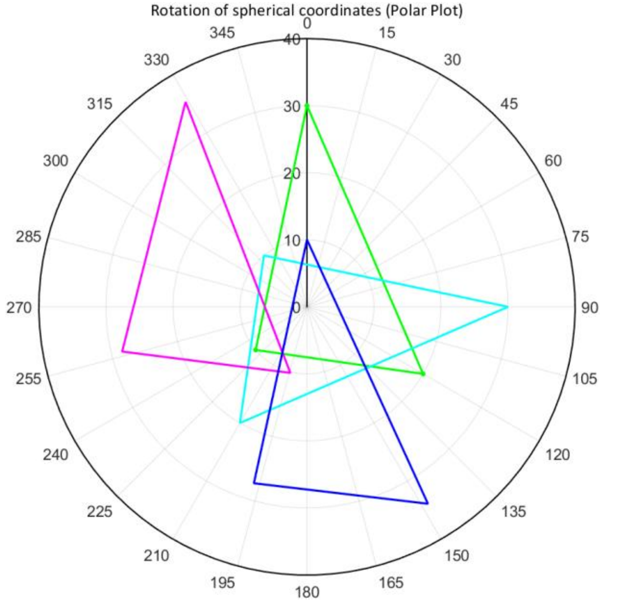
```
v1, v2 = two known vectors defining a plane
% Normal vector to this plane
v_norm = cross(v1, v2)';
[az, el] = Cart2Sph(v_norm);
xyz;  % ← make sure offset to origin before rotations

%--- Rotates along azimuth to bring back along RotY, followed by rotation along elevation to bring it coplanar
xyz_ = RotY(el - 90) * RotZ(-az) * xyz;  ← In that order
```

Spherical rotations

```
function [lambda_out, phi_out] = RotX_sph(lambda, phi, alpha)
% Rotation by angle α about X-axis

    lambda_out = atan2d(sind(lambda) * cosd(alpha) - tand(phi) * sind(alpha), cosd(lambda));  % λ’
    phi_out = 90 - acosd(sind(phi) * cosd(alpha) + cosd(phi) .* sind(lambda) * sind(alpha));  % ϕ’

end

function [lambda_out, phi_out] = RotY_sph(lambda, phi, alpha)
% Rotation by angle α about Y-axis

    lambda_out = atan2d(sind(lambda), cosd(lambda) * cosd(alpha) + tand(phi) * sind(alpha));  % λ’
    phi_out = 90 - acosd(sind(phi) * cosd(alpha) - cosd(phi) .* cosd(lambda) * sind(alpha));  % ϕ’

end

function [lambda_out, phi_out] = RotZ_sph(lambda, phi, alpha)
% Rotation by angle α about Z-axis

    lambda_out = lambda + alpha;  % λ’ = λ + α
    phi_out = phi;  % ϕ’ = ϕ

end
```

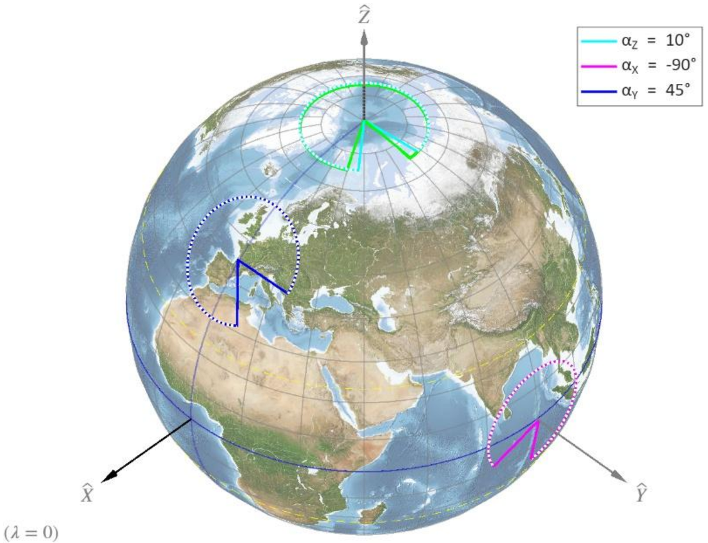
Spherical rotations

```
%--- 0) Reference spherical path
phi = [0, 120, 230 0];      % [°] φ Azimuthal angle ≡ Azimuth α  ≡ Longitude λ
theta = [30, 20, 10, 30];  % [°] λ Longitudinal angle
lambda = 90 - theta;       % [°] θ Polar angle ≡ 90 – Elevation δ ≡ 90 – Latitude ϕ ≡ colatitude or polar angle
xyz = Sph2Cart(phi, lambda, 1 + 0*phi);
polarplot(RAD(phi), theta, 'g-');

%--- 1)  Rotation by angle α about Z-axis
alpha_Z = 90;
% 1.1) Using cartesian coordinates
xyz_ = RotZ(alpha_Z) * xyz;
[phi_, lambda_] = Cart2Sph(xyz_(1,:), xyz_(2,:), xyz_(3,:));
polarplot(RAD(phi_), 90 - lambda_, 'c--');
% 1.2) Using direct spherical rotation
[phi_, theta_] = RotZ_sph(phi, theta, alpha_Z);
polarplot(RAD(phi_), theta_, 'c-');

%--- 2) Rotation by angle α about X-axis
alpha_X = 20;
% 2.1) Using cartesian coordinates
xyz_ = RotX(alpha_X) * xyz;
[phi_, lambda_] = Cart2Sph(xyz_(1,:), xyz_(2,:), xyz_(3,:));
polarplot(RAD(phi_), 90 - lambda_, 'm--');
% 2.2) Using direct spherical rotation
[phi_, theta_] = RotX_sph(phi, theta, alpha_X);
polarplot(RAD(phi_), theta_, 'm-');
% → appears like a vertical shift in polar plot
```



3D rotations: cartesian & spherical

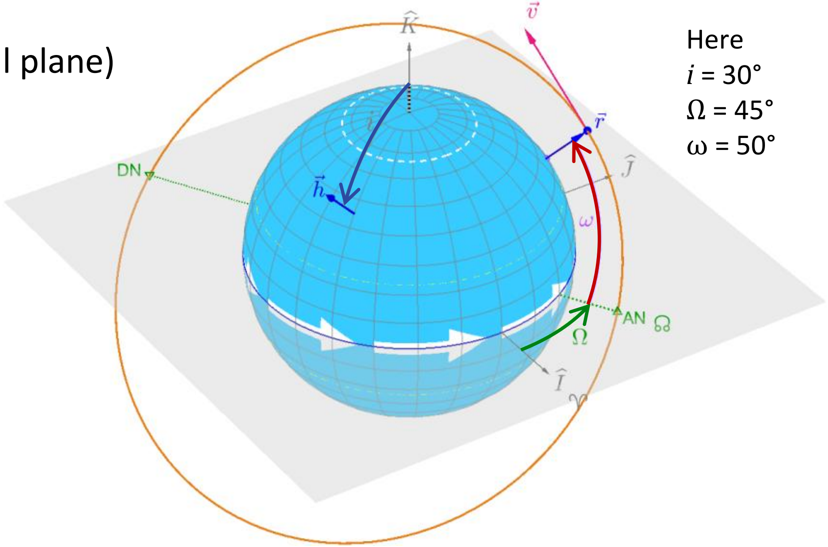
```
% Reference small circle (with a dent for reference)
[lat, lon] = SmallCircle(90, 0, 15, [0:300]);
lat = [lat, 90, lat(1)];
lon = [lon, 0, lon(1)];
xyz = Sph2Cart(lon, lat, Earth.r1_km * 1.01);
Plot3(xyz, 'g-');

%--- 1) Rotation by angle α about Z-axis
alpha_Z = 10; % (counter-clockwise)

% 1.1) Cartesian rotation
xyz_ = RotZ(alpha_Z) * xyz;
Plot3(xyz_, 'c--');

% 1.2) Spherical rotation
[lon_, lat_] = RotZ_sph(lon, lat, alpha_Z);
xyz_ = Sph2Cart(lon_, lat_, Earth.r1_km * 1.01);
ph_Z = Plot3(xyz_, 'c-');

% 1.3) Rotated small circle
[lat_, lon_] = SmallCircle(0, 90, 15, [0:300] + alpha_Z);
xyz_ = Sph2Cart(lon_, lat_, Earth.r1_km * 1.01);
Plot3(xyz_, 'w:');

%% (continued)
```



Rotations (continued)

- Coordinate transformation matrix: Perifocal → ECI
- Counter-clockwise rotations (right-hand)
  1) `RotZ(ω)` rotation by $\omega°$ over $\widehat{K}$ ($\widehat{W}$ in perifocal plane)
  2) `RotX(i)` followed by $i°$ over $\hat{I}$ axis
  3) `RotZ(Ω)` followed by $\Omega°$ over $\widehat{K}$ axis

  in that order.



Here
i = 30°
Ω = 45°
ω = 50°

Main rotation operator:

```
Orbit.RotPeri2ECI = RotZ(Orbit.RAAN) * RotX(Orbit.i) * RotZ(Orbit.w);
```

SOL Part II – Perifocal

Rotation along a given axis

```
[x, y, z] = peaks(20);  % 3× [20, 20]
z = z / 3;

% reshapes the n×n×n into [3×m]
XYZ = [x(:), y(:), z(:)]';

[XYZ_, R, t] = AxelRot(XYZ, 25, direction, []);
%size(XYZ_)

Figure(2);
axis equal;
surf(reshape(XYZ_(1, :),20,20), reshape(XYZ_(2, :),20,20), reshape(XYZ_(3, :),20,20));
X_label("x-axis");
Y_label("y-axis");
Z_label("z-axis");
view(3);

[XYZ_, R, t] = AxelRot([[0,0]; Lims(y(:)); [0,0]], 25, direction, []);
Plot3(XYZ_, 'r--', 'Linewidth', 1.5);
```

AxelRot, J2000_to_epoch

Rotation along a given axis (continued)

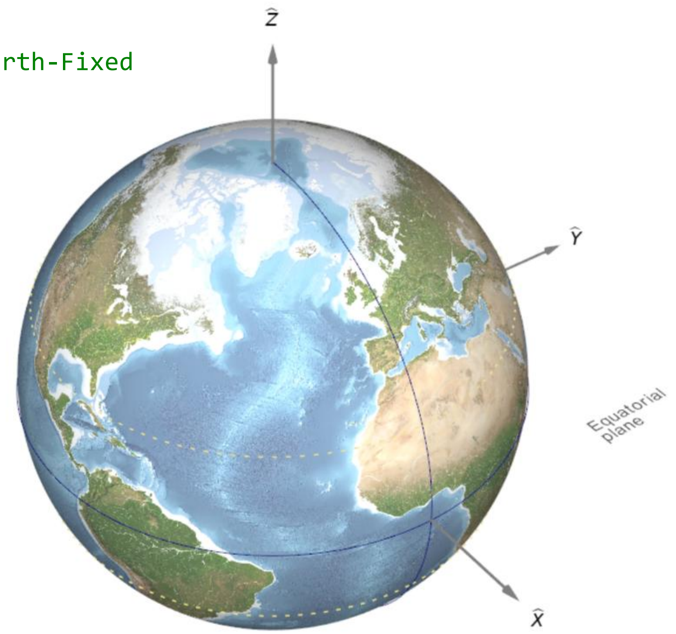
- Example of rotation of stars on the Celestial Sphere

```
% → See SOL_Celestial_Sphere.m
theta = 360 * (JulianDate(Date_UTC)-Earth.Epoch_J2000)/Earth.Year_days / Earth.EquinoxPrecession_period; % [°]
if 1  % A) decomposition into 3 rotations
    [RA_, Dec_] = RotX_sph(Stars.RA, Stars.Dec, Earth.NEP.Dec - 90);  %  a) bring to NEP axis
    [RA_, Dec_] = RotZ_sph(RA_, Dec_, theta);                         %  b) apply rotation on Z axis
    [Stars.RA, Stars.Dec] = RotX_sph(RA_, Dec_, 90 - Earth.NEP.Dec);  % a’) back to original axis
else  % B) equivalent with AxelRot:
    p1 = [0;0;0];  % Center of the Earth
    p2 = Sph2Cart(Earth.NEP.RA, Earth.NEP.Dec);  % NEP (normalized)
    u = p2 - p1;  % Rotation axis
    XYZ = Sph2Cart(Stars.RA, Stars.Dec);
    [XYZ_] = AxelRot(XYZ, theta, u, []);
    [Stars.RA, Stars.Dec] = Cart2Sph(XYZ_);
end
```

Recap exercise: Generate the following figures

0) Draw Earth in ECEF as starting point

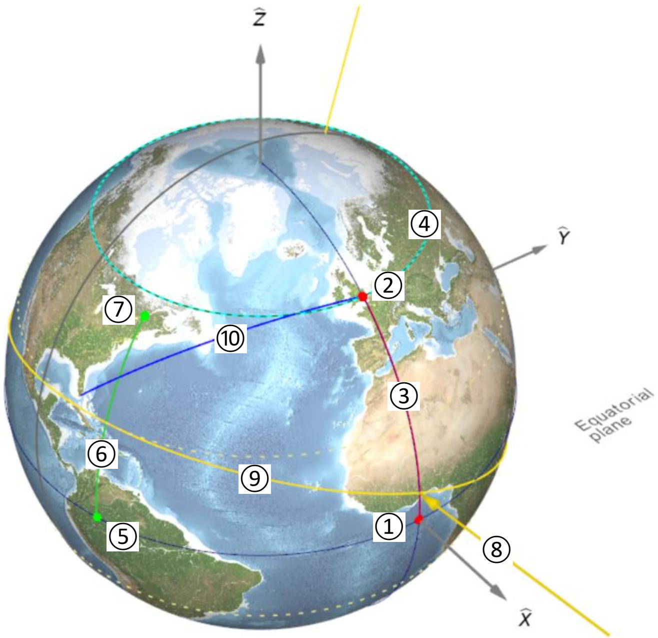
```
ViewMode = 'ECEF'    # Earth-Centered, Earth-Fixed
pl = DrawEarth3D(pl, ViewMode)
```



Recap exercise: Generate the following figure

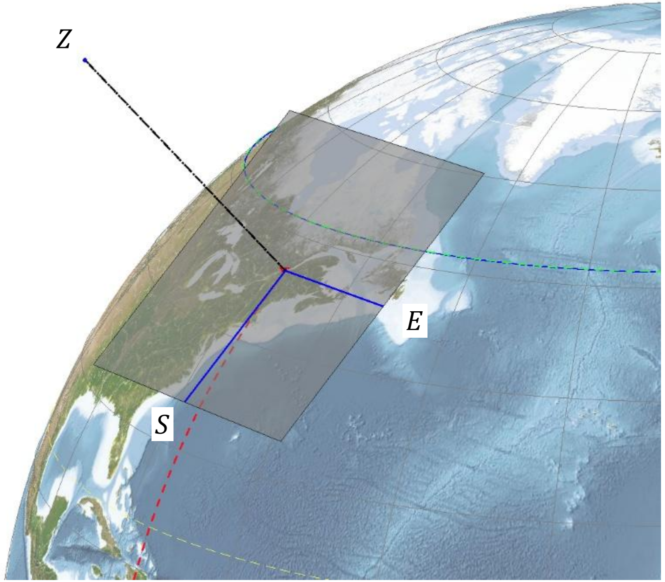
1) Draw starting point at $\lambda = 0°, \varphi = 0°$
2) Draw arc line up to Greenwich $\lambda = 0°, \varphi = 51.4934°$
3) Draw arc line up to Greenwich
4) Draw Earth Parallel at Greenwich latitude
5) Compute point location westward to longitude λ=71.2778°W (Quebec City)
6) Draw arc from Equator
7) Draw last point of arc
8) Draw Sun vector pointing to Earth center,
   knowing Sun longitude $\lambda_{\odot}= -0.1614°$, and Sun declination $\delta_{\odot}$=7.4364°
9) Draw Ecliptic line knowing Earth obliquity ϵ and GMST angle at epoch
10) Draw Great Circle from Miami to Greenwich

→ See code `SOL_Rotations.py`



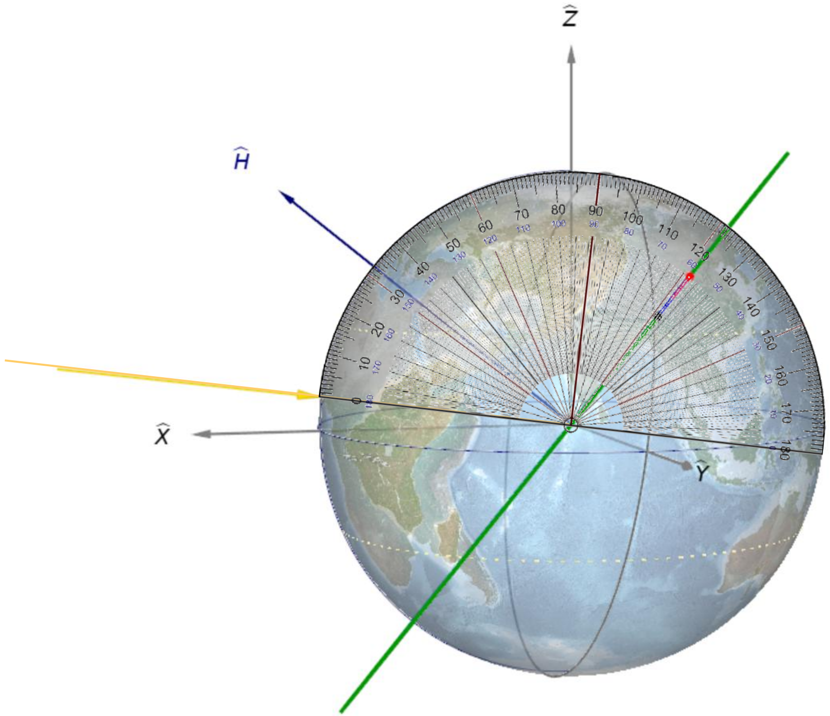
Draw topocentric SEZ Plane

- Draw South/East/Z (up) plane at Laval University location
  - Ground Station/Observer "Quebec City": λ = -71.357°, ϕ = 46.774



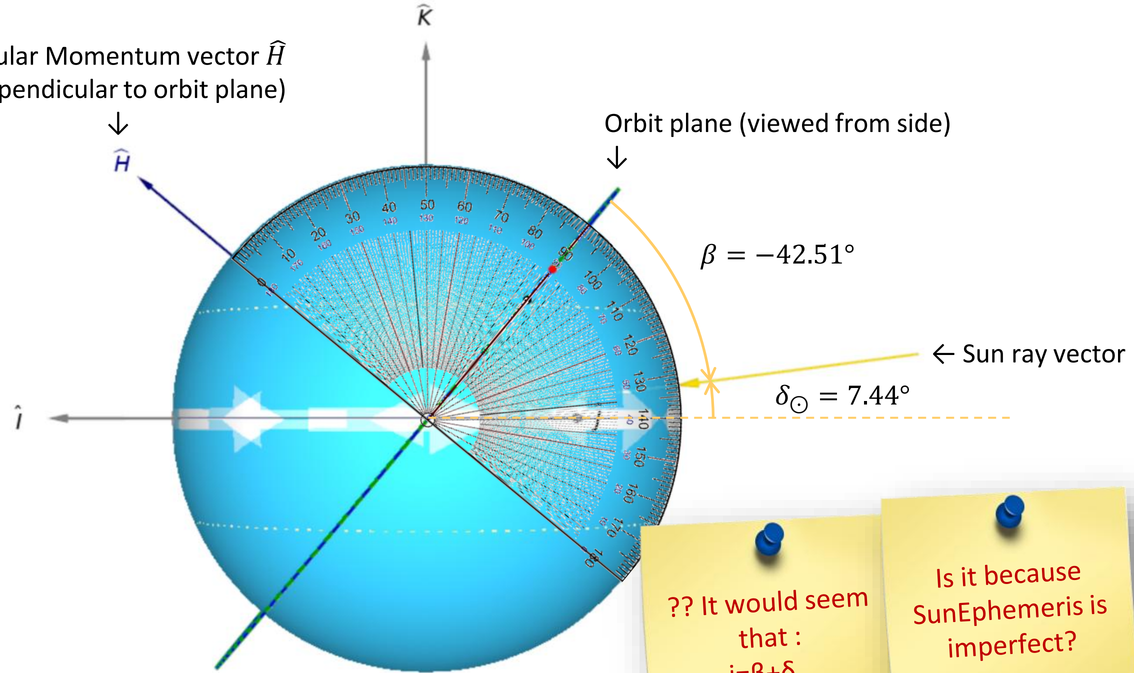
B angle (after Orbit propagation)

- ➔ See III-323



β angle (present after Orbit propagation)

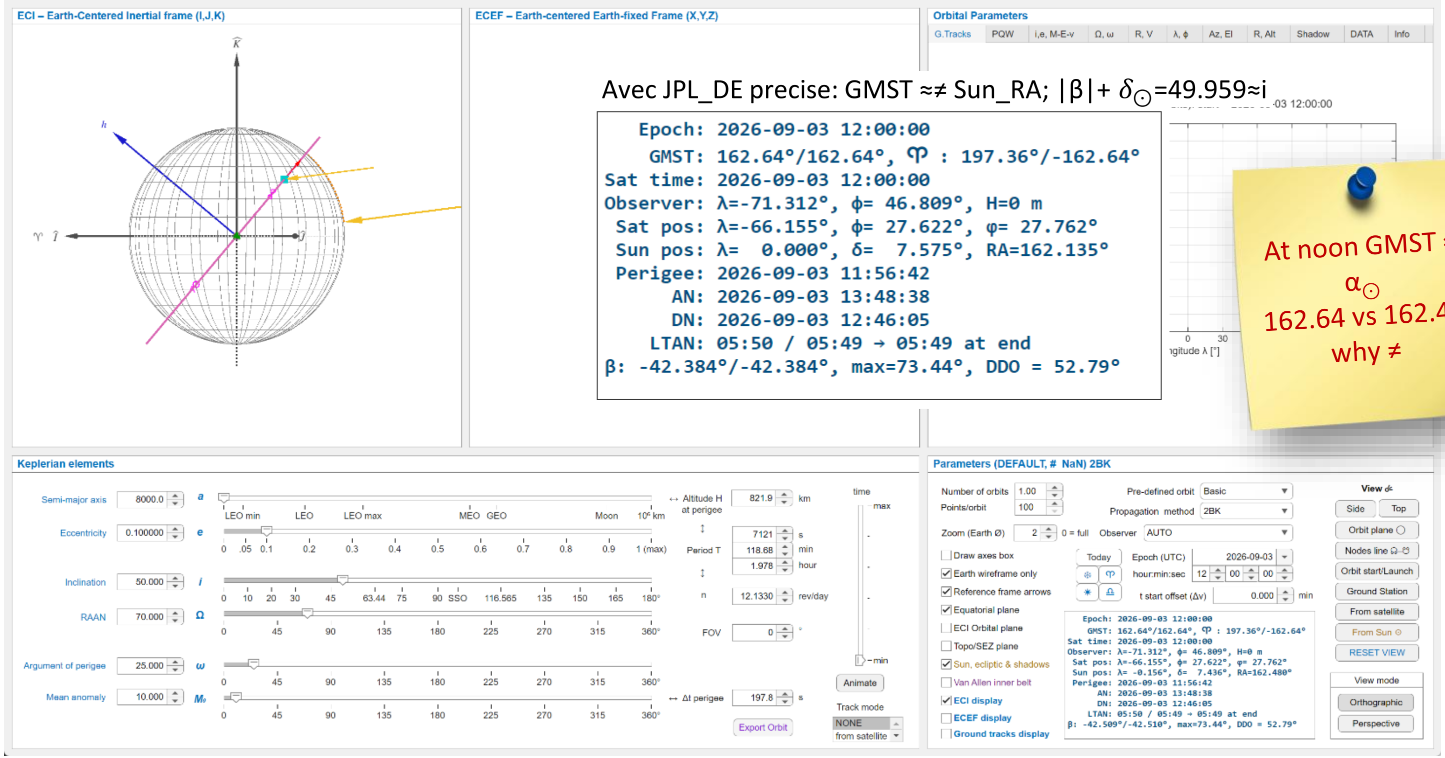
- ((identify those affecting β))
  - f(Ω–α), δ + hour
- COE:
  - a = 8000 km (independent of β)
  - e = 0.1
  - **i = 50°**
  - **Ω = 70°**
  - ω = 25° (independent of β)
  - M_o = 10° (independent of β)



?? It would seem that :
i=β+$\delta_{\odot}$
50 = 42.51 + 7.44

Is it because SunEphemeris is imperfect?

- When $i = 0°, \beta \equiv \delta_{\odot}$
  - Because: β = asind(cosd(δ) sind(i) sind(Ω - α) + sind(δ) cosd(i))
    @i≡0 → β = asind(0 + sind(δ) × 1) → β≡δ



Avec JPL_DE precise: GMST ≈≠ Sun_RA; |β|+ $\delta_\odot$=49.959≈i

```
    Epoch: 2026-09-03 12:00:00
     GMST: 162.64°/162.64°, ♈ : 197.36°/-162.64°
 Sat time: 2026-09-03 12:00:00
 Observer: λ=-71.312°, ϕ= 46.809°, H=0 m
  Sat pos: λ=-66.155°, ϕ= 27.622°, φ= 27.762°
  Sun pos: λ=  0.000°, δ=  7.575°, RA=162.135°
  Perigee: 2026-09-03 11:56:42
       AN: 2026-09-03 13:48:38
       DN: 2026-09-03 12:46:05
     LTAN: 05:50 / 05:49 → 05:49 at end
β: -42.384°/-42.384°, max=73.44°, DDO = 52.79°
```

At noon GMST ≈ $\alpha_\odot$
162.64 vs 162.48
why ≠

Solution

```
Observer = DefineObserverLocation('Quebec');

% Observer plane dimension
SEZ_r = 1500; % plane radius
SEZ_H = 2 * SEZ_r; % [km] % Height of local Z axis

Rot = RotZ(Observer.Lon) * RotY(-Observer.Lat);

% Defines vertices of the SEZ plane (unroated)
if 1 % square
    x = [1, 1, -1, -1, 1];
    y = [-1, 1, 1, -1, -1];
else % circle
    [x, y] = Circle(0, 0, 1);
end
EqPlane_vertices = [x; y; 0 * x];
```

```
% Draws Topo/SEZ plane
SEZ_plane = [repmat(Earth.r1_km, 1, size(EqPlane_vertices, 2)); ...
             SEZ_r * EqPlane_vertices([1 2], :)]; % Touches Earth's surface
SEZ_plane = Rot * SEZ_plane;
Observer.SEZ.plane = fill3(SEZ_plane(1,:), SEZ_plane(2,:), SEZ_plane(3,:), 'k’);
Observer.SEZ.plane.FaceColor = 0.5 * [1,1,1];
Observer.SEZ.plane.FaceAlpha = 0.6; % smaller value is more transparent

% Draws Reference axes S (South), E (East), Z (Up)
SEZ_frame = [Earth.r1_km + [0,0,0, SEZ_H]; 0, 0 SEZ_r 0; 0, -SEZ_r 0 0];
SEZ_frame = Rot * SEZ_frame;
Observer.SEZ.S = Plot3(SEZ_frame(:,[1,2]), 'b-');
Observer.SEZ.E = Plot3(SEZ_frame(:,[1,3]), 'b-');
Observer.SEZ.Z = Plot3(SEZ_frame(:,[1,4]), 'k--');
```

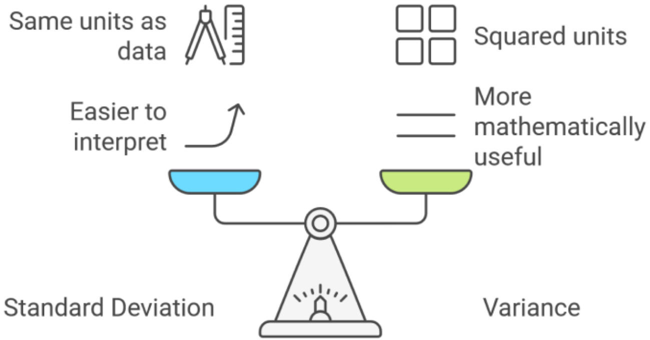
Statistics

- Variance vs Standard Deviation
- Normal distribution – Gaussian profile

Variance vs Standard Deviation

- Variance and standard deviation are both measures of data dispersion, but they differ in how they are calculated and what information they provide.
- Variance represents the average squared difference of data points from the mean
- Standard Deviation is the square root of the variance and is expressed in the same units as the original data.

$$\mathrm{std} = \sqrt{\mathrm{var}} = \sigma$$
$$\mathrm{var} = \sigma^2$$



https://6sigma.us

Normal distribution – Gaussian profile

$$f(x) = A\, e^{\left(-\frac{1}{2}\frac{x-\mu}{\sigma}\right)^2}$$

μ = distribution center

σ = half-width @ $\sqrt{1/e}$

= standard deviation

$S = A\sqrt{2\pi}\,\sigma$ = surface area

$$f(x) = A\, e^{\left(-\frac{x-\mu}{w}\right)^2}$$

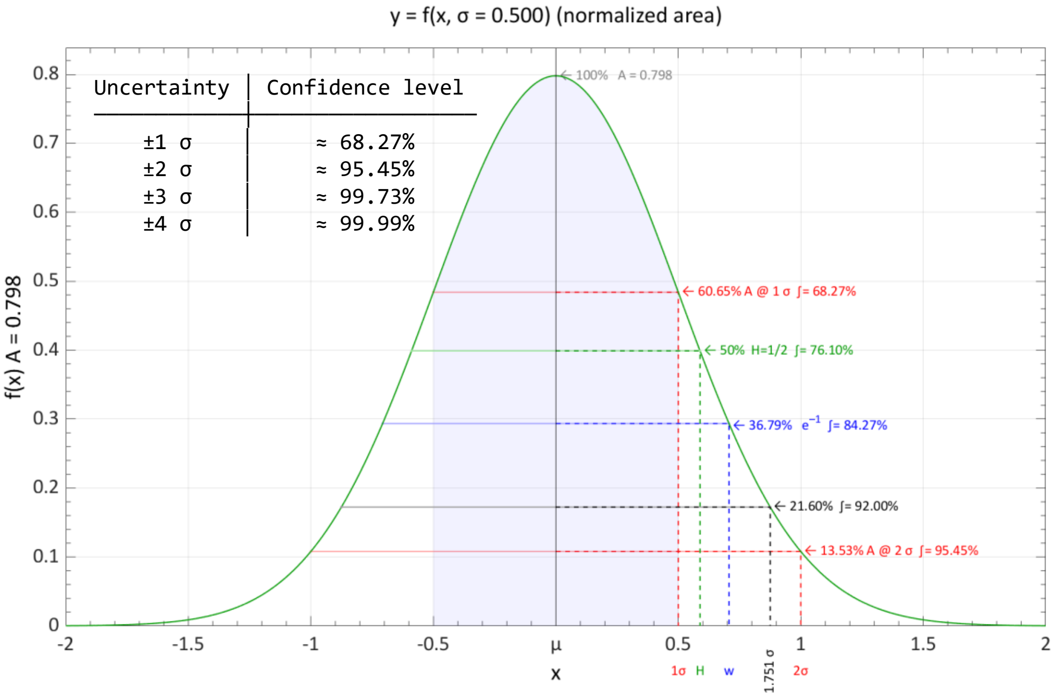
w = half-width @ $1/e$

= $\sqrt{2\pi}\,\sigma$

$S = A\sqrt{\pi}\,w$ = surface area

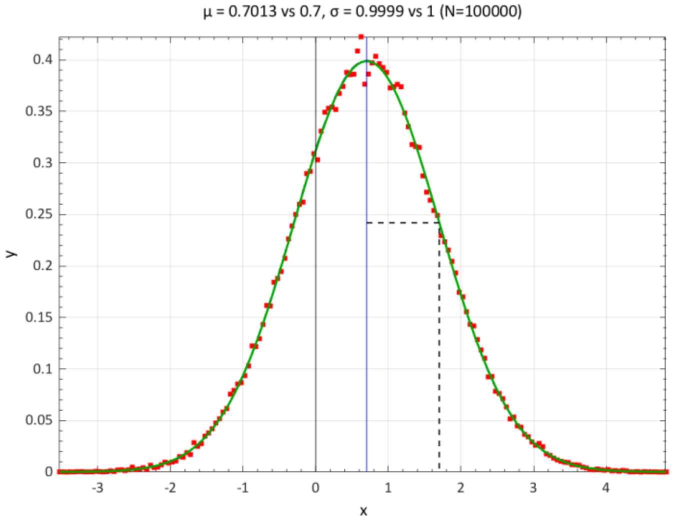
$$-\infty < x < \infty$$

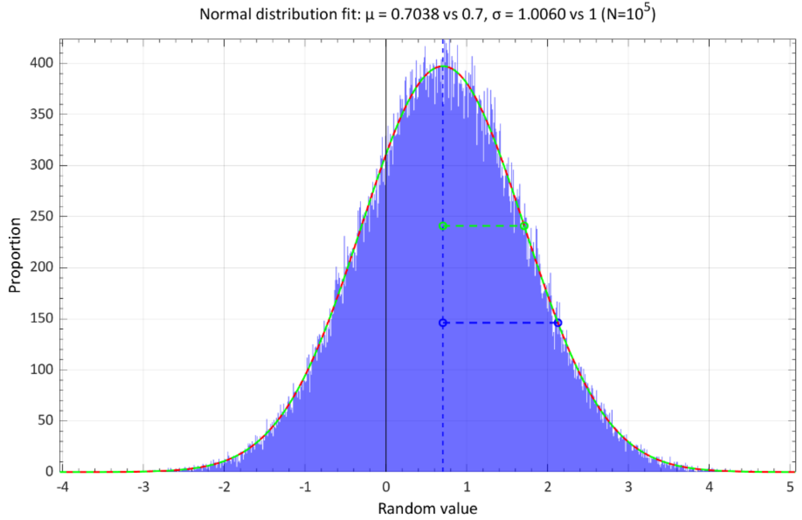
Surface from $\pm x = A\,\mathrm{erf}\left(\frac{x}{\sqrt{2}\sigma}\right)$



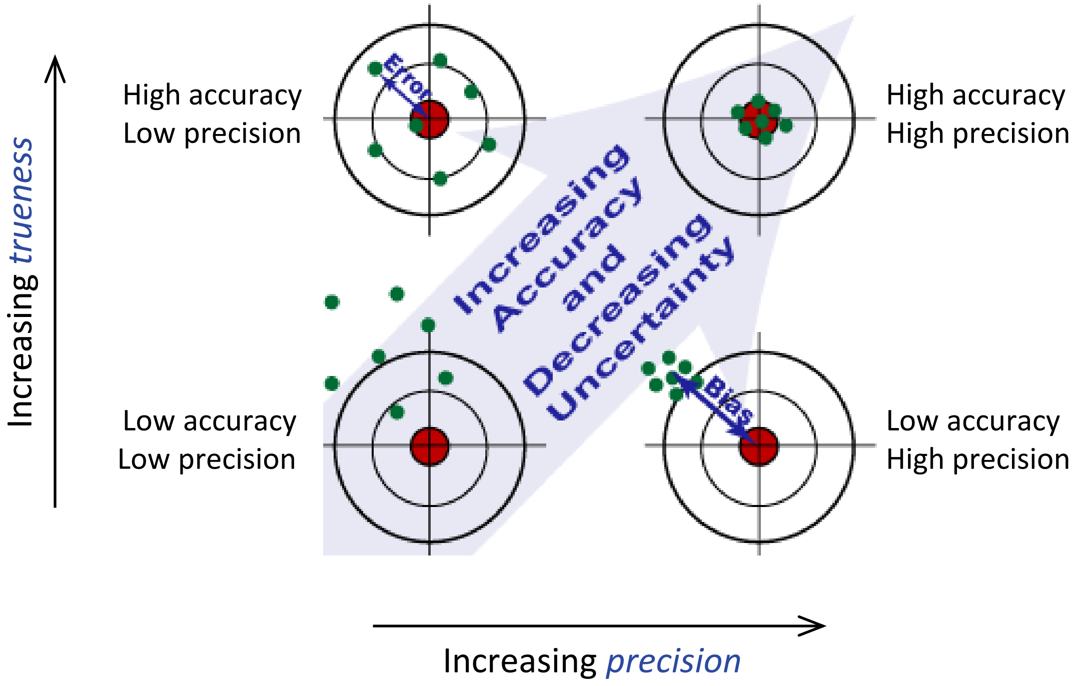
Normally distributed pseudorandom numbers

```
N = 10^5;
mu = 0.7;
sigma = 1;  % (arbitrary)
p = sigma * randn(1,N) + mu;
histcounts(p)
```





Accuracy vs precision



Accuracy vs precision

- For a long-range shooting, is it preferable to have a high accuracy *or* high precision rifle?

For long-range shooting, **both accuracy and precision are required**, but if forced to choose, **precision is the more fundamental requirement**.

1 Definitions (technically distinct)

- **Accuracy**: how close shots land to the true point of aim.
- **Precision**: how tightly grouped the shots are (repeatability).

A rifle can be:

- Precise but not accurate → tight group, but offset.
- Accurate but not precise → average impact near target, but wide spread.
- Neither → unacceptable for long range.

2 Why precision is more critical

At long range:

- Environmental corrections (wind, drop, Coriolis, etc.) can be calculated and adjusted.
- Scope turrets can shift the point of impact.

If the rifle is **precise**, even if initially inaccurate:

$$\text{You can zero it.}$$

You simply dial or hold corrections to move the tight group onto the target.

However:

If the rifle lacks precision:

- The shot dispersion may exceed the target size.
- No correction can fix random spread.

In other words:

> Accuracy can be corrected.
> Lack of precision cannot.

3 Long-range implication

At 1,000 m, a 1 MOA rifle produces:

$$\approx 29 \text{ cm } (11.5 \text{ in}) \text{ spread}$$

If the rifle is inconsistent, that spread grows — and ballistic corrections become irrelevant.

Thus professional long-range shooters prioritize:

- Mechanical precision (barrel quality, bedding, ammunition consistency)
- Repeatability

Then they adjust for accuracy.

Accuracy can be corrected.
Lack of precision cannot.

→ For long-range shooting, high precision is more essential than initial accuracy, because a consistent rifle can always be zeroed, whereas an imprecise rifle cannot be corrected to reliably hit distant targets.

General programming notes

Use of structures

- Use structures whenever possible

```
[d, e, f, g] = fnc(a, b, c)
```

→ preferred:

```
out = fnc(in)
```

with:

```
in.a, in.b, in.c
out.d, out.e, out.f
```

```
ColRA = 1;
StarCat_J2000(:,ColRA) → Stars.RA
```

→ Easy to manage, remove or add elements

(Vallado)

D.6 Computer Programming

In this book we focus mainly on presenting workable astrodynamic routines through computer programming. Many organizations shy away from computer programming and therefore do little with it. This section documents *sound* and *tested* programming practices which, if applied rigorously in *any* language, will result in code that is more efficient, faster running, and more accurate. They will also greatly reduce the lifecycle costs for maintaining this computer code.

Engineers must firmly grasp good programming techniques to correct problems in existing code. Much of today's code was developed decades ago when documentation and efficiency weren't as important. In fact, many of the original programmers are no longer alive, survived only by their code. Fortunately (or unfortunately), much of the code still works, but continual fixes to huge, ill-conceived programs have begun to create problems. Examples include inadvertent launches of bomber aircraft in anticipation of nuclear war (NORAD 1979), loss of life in accidental operations, and economic and financial disasters like the Vancouver Stock exchange in 1974. Whatever the case, computer code is very important, and it demands knowledge and care.

Most astrodynamics code is in some form of FORTRAN. Although the code usually works, you'll quickly see the horrors of programming legacy whenever you need to change it. You'll need to learn every version of FORTRAN, tricks to make a program think it is using characters, and so on. The most common excuse for this mess is that we don't have enough time and money to fix the problem. Weak as this argument is, it has stood up, and only a huge loss of life may be able to change it. Competent engineers will be prepared for that time.

Use of structures: Example

```
Earth = AstroConstants();

struct with fields:
             mu: 398600.4418
        mass_kg: 5.97216849407428e+24
             AU: 149597870.7
              a: 149598023
              e: 0.0166977045436129
          r1_km: 6378.137
          r2_km: 6356.75231424518
      radius_km: 6371.00877141506
            ecc: 0.0818191908426216
           ecc_: 0.0820944379496946
              f: 0.00335281066474746
             g0: 9.80665
              g: 9.797643222

           etc.
```

(see notes in function)

← Maybe too many significant digits… (from algebraic calculations)

General programming notes

- Whenever possible, use different formulas or multiple algorithms to validate calculations → *SAVES LIVES*!
- For clarity write:
  - instead of: `86400` write: `24*60^2 % [s/day]`
  - instead of: `86164.0905` write: `Earth.sidereal_period_s % [s]`
  - instead of: `angle = hours/15` write: `angle = hours/24 * 360 % [°]`
  - instead of: `scale = 0.17 * value` write: `scale = 17/100 * value % ← clear indication of %`
  - instead of: `P = 1.658e-4 * a^(3/2)` write: `P = 2*pi * sqrt(a^3 / mu_Earth) / 60 % [minutes]`
- Angles can be expressed in either degrees [0–360° / ±180°] or hours [2–24 hr]
  - θ [hr] = 24 × θ [°] / 360° or θ [°] = 350 × θ [hr] / 24°
- Avoid hard-coding constants or duplicating code
- Quite often found in my code:

```
if 0 % STUDY/CHECK
    (some code here)
end
```

→ to enclose unused study check points for checking values or evaluating alternative methods

Comments

- Write in "telegraphic English" with a minimum of articles
  - Earth's trajectory / Earth trajectory
  - ~~The~~ Earth trajectory

- First vs 2nd person
  - Descriptive: This code **does** this
  - Command: Do this

Time handling

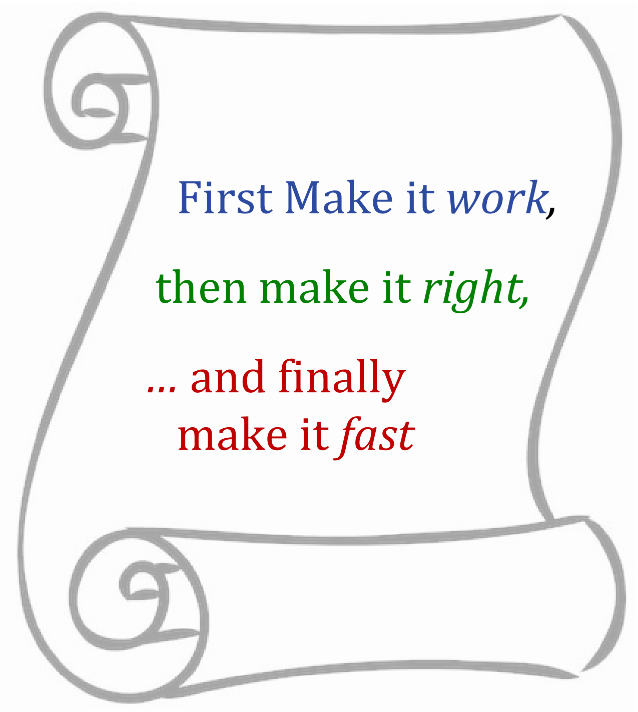
- How to write dates?
  - Using SI: **yyyy-mm-dd**, **HH:MM:SS** (24 h format) ISO
  - vs American time is confusing: DD/MM/YYYY → Use June 02, 2025 to avoid confusion

→ Programming with time

- **`deg_to_DMS()`** & **`DMS_to_deg()`**
  - Ex: `deg_to_DMS([-16, 42, 58]) → -16.71611 = -deg_to_DMS([16, 42, 58])`
    `≠ -16 + 42/60 + 58/60^2 → ≡ -(16 + 42/60 + 58/60^2)`

- Handling of Leap Years and the Gregorian Calendar

Good programming habits



- Software development principle, that prioritizes stages of the development process:
  - first get the code functioning,
  - then restructure or reorganize it for quality and correctness,
  - and finally optimize it for speed, *only if and when necessary*.
- This approach avoids premature optimization, ensuring that time is not wasted on improving the performance of code that is not yet proven to be correct or even necessary.

Handling images as arrays

Indexing: arrays, images, georeferenced maps

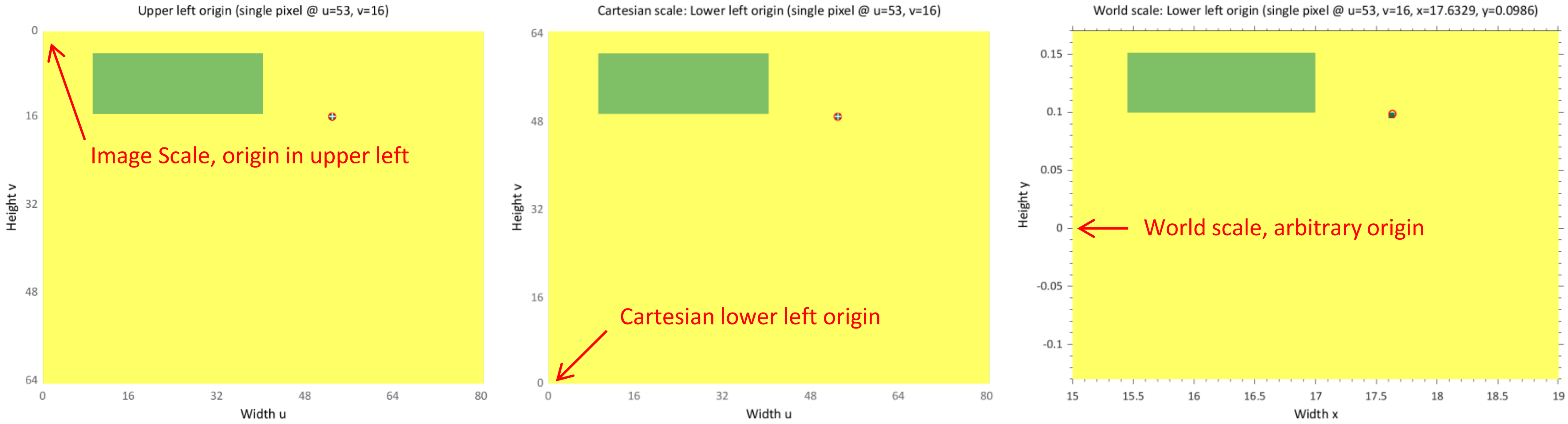
1) Matrix indexing & Image pixel scale

mat(v, u)

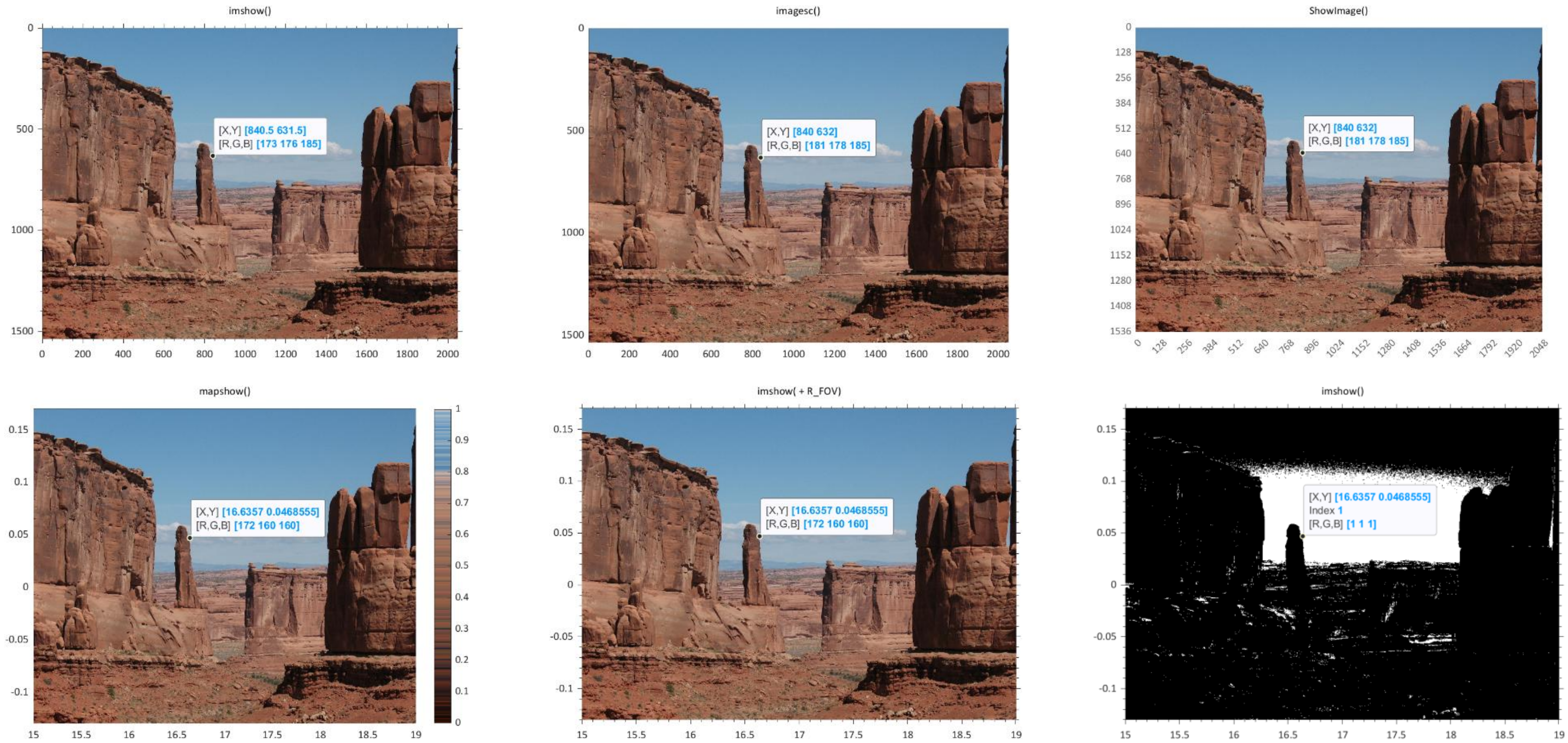
v=1 → top row

2) Cartesian figure scale

3) World scaled reference



The art of image display

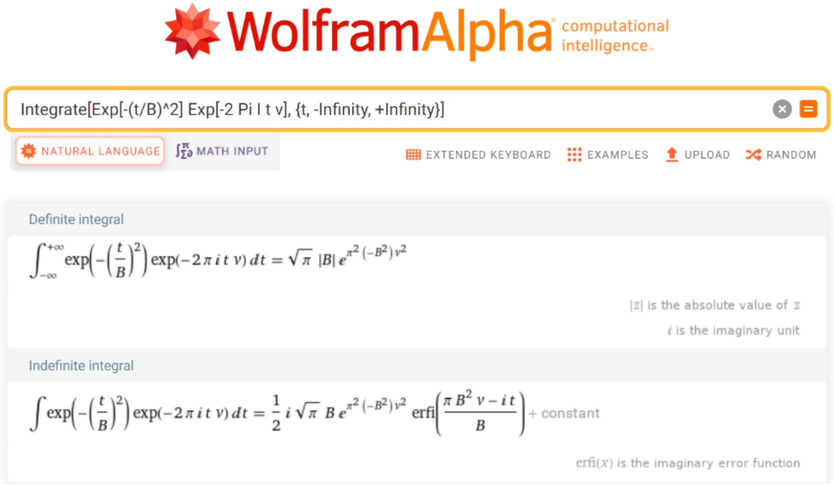


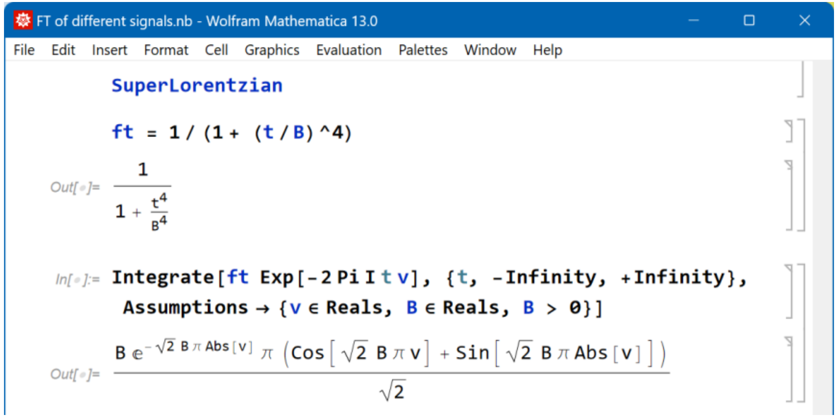
Validation & pitfalls

- Validation with algebraic solutions
- Errors even in some reverence books
- Check validity range of provided equations

Validation with algebraic solutions

- www.WolframAlpha.com / Mathematica





- Also MATLAB symbolic toolbox
  - `sym('x')`

Do not blindly trust all information sources!

- e.g. books, publications, internet, etc.
- Example: Error in Prussing 1993, Eq. (3.9b)
  - Orbital velocity equation:

$$v_x = -\frac{\mu}{h}\left[\cos(\Omega)\left(\sin(\theta) + e\,\sin(\omega)\right) + \sin(\Omega)\left(\cos(\theta) + e\,\cos(\omega)\right) \cdot \cos(i)\right]$$

$$v_y = -\frac{\mu}{h}\left[\sin(\Omega)\left(\sin(\theta) + e\,\sin(\omega)\right) - \cos(\Omega)\left(\cos(\theta) + e\,\cos(\omega)\right) \cdot \cos(i)\right]$$

$$v_z = +\frac{\mu}{h}\left(\cos(\theta) + e\,\cos(\omega)\right) \cdot \sin(i)$$

→ Correct equation in Battin 1999 Problem 3-21 and Chobotov (4.234)

- Example: Error in Vallado 2014, Eq. (11-25)

$$\mathcal{P}_{\Omega} = \mathcal{P}_{K}\left[\frac{1}{1 + \frac{3J_2}{4}\left(\frac{R_{\oplus}}{p}\right)^2\left\{\sqrt{1-e^2}\left[(2 - 3\sin^2(i)) + (4 - 5\sin^2(i))\right]\right\}}\right] \qquad (11\text{-}25)$$

→ *Always counter-check your calculations with equivalent formulas or known reference cases*

Some equations are valid only in specific situations

e.g.: www.braeunig.us/space/orbmech.htm#launch

Orbit Tilt, Rotation and Orientation

Above we determined the size and shape of the orbit, but to determine the orientation of the orbit in space, we must know the latitude and longitude and the heading of the space vehicle at burnout.

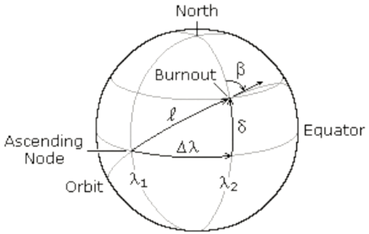


Figure 4.9

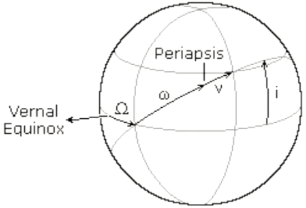


Figure 4.10

Figure 4.9 above illustrates the location of a space vehicle at engine burnout, or orbit insertion. β is the azimuth heading measured in degrees clockwise from north, δ is the geocentric latitude (or declination) of the burnout point, $\Delta\lambda$ is the angular distance between the ascending node and the burnout point measured in the equatorial plane, and ℓ is the angular distance between the ascending node and the burnout point measured in the orbital plane. λ_1 and λ_2 are the geographical longitudes of the ascending node and the burnout point at the instant of engine burnout. Figure 4.10 pictures the orbital elements, where i is the inclination, Ω is the longitude at the ascending node, ω is the argument of periapsis, and ν is the true anomaly.

If β, δ, and λ_2 are given, the other values can be calculated from the following relationships:

(4.33) $$\cos i = \cos\delta \sin\beta$$

(4.34) $$\tan \ell = \frac{\tan\delta}{\cos\beta}$$

(4.35) $$\boxed{\tan\Delta\lambda = \sin\delta \tan\beta}$$

OK, but only valid $0° < \beta < 90°$; use instead $\Delta\lambda = \operatorname{acos}\left(\frac{\cos(\beta)}{\sin(i)}\right)$ from 2nd spherical law of cosines

(4.36) $$\omega = \ell - \nu$$

(4.37) $$\lambda_1 = \lambda_2 - \Delta\lambda$$

Understand the implied geometry

- Fortescue 2011, Eq (5.16) should be a negative sign

The Earth rotates through one revolution in its *sidereal* period of τ_E, where $\tau_E = 86\,164.1$ s, neglecting small secular variations. This sidereal period of 23 h 56 min is with respect to the stars. The rotation period with respect to the Sun is of course the mean solar day of 24 h. If the satellite's nodal period is τ, then the contributions to $\Delta\phi$, which is caused by the Earth's rotation, will be given by

$$\Delta\phi_1 = -2\pi\frac{\tau}{\tau_E} \text{ rad/orbit} \tag{5.14}$$

The regression of the line of nodes (equation 4.37) contributes

$$\Delta\phi_2 = -\frac{3\pi J_2 R_E^2 \cos i}{a^2(1-e^2)^2} \text{ rad/orbit} \tag{5.15}$$

124 *MISSION ANALYSIS*

The total increase in longitude at the equator is

$$\Delta\phi = \Delta\phi_1 + \Delta\phi_2 \text{ rad/orbit} \tag{5.16}$$

Verifying equations through sources

- Vallado 2013: p. 579:

Many of the same factors affecting atmospheric-density calculations (27-day, diurnal, etc.) also affect the magnitude of solar-radiation pressure. Wertz (1978:130) gives a time-varying formula instead of the constant value, accounting for the variation over a year (notice the different solar irradiance value):

$$SF = \frac{1358}{1.004 + 0.0334\cos(D_{aphelion})}\frac{\text{W}}{\text{m}^2} \qquad (8\text{-}42)$$

- Wertz 1978, p.130:

5.3.1 Solar Radiation

The mean solar energy flux integrated over all wavelengths is proportional to the inverse square of the distance from the Sun. To within 0.3%, the mean integrated energy flux *at the Earth's position* is given by:

$$F_e = \frac{1358}{1.0004 + 0.0334\cos D}\,\text{W/m}^2 \qquad (5\text{-}29)$$

where 1358 W/m^2 is the mean flux at 1 AU, and the denominator is a correction for the true Earth distance. D is the "phase" of year, measured from July 4, the day of Earth aphelion [Smith and Gottlieb, 1974]. This is equivalent to a mean

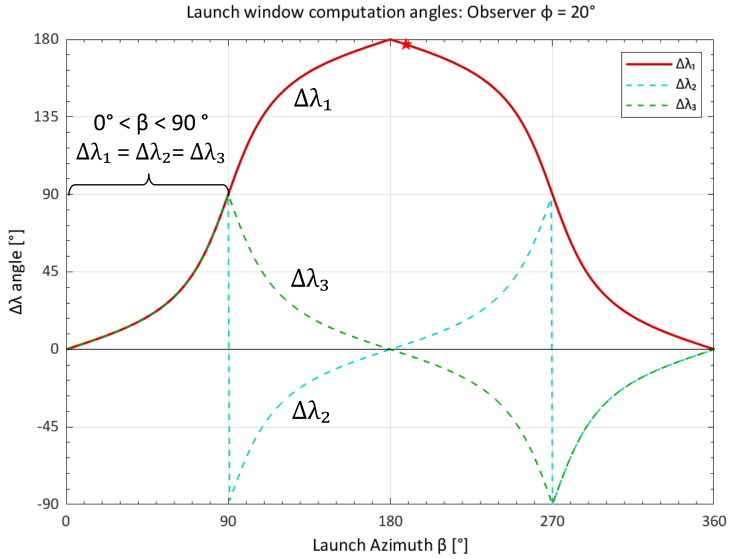
Study of different formulas

$$i = \text{acos}(\sin(\beta)\cos(\phi))$$

$\Delta\lambda_1 = \text{atan}(\sin(\varphi)\tan(\beta))$ (Braeunig: phase shift @90°)

$\Delta\lambda_2 = \text{acos}(\cos(\beta)/\sin(i))$ ← better for wider range (quadrant automatically handled by trig function)

$\Delta\lambda_3 = \Re\{\text{asin}(\tan(90-i)\tan(\varphi))\}$ (numerical complex value @270°)



IMP: Visual study helps better understand formulas;

→ be sure to properly understand the behavior in different quadrants, in order to *use the appropriate functions* or *apply manual quadrant correction* to ensure getting correct results

Launch Windows

Importance to test code

- Proper handling of trig functions is delicate → can even be dangerous if not!
  - Examples of spacecraft failure due to “software bugs”, like trig phase shifts.
- Important to thoroughly check algorithms and code with examples covering *all of applicable cases*
  - E.g. not enough to check inclination angles between 0..90° → do not forget 90°..180° (including SSO ≈ 98°)

→ Whole philosophy on code testing…

Some typography

Use of quotes and double quotes

- Be careful not to overuse “double quotes” when not needed → use *italics* instead
  - A literal quote by someone: Einstein said: “I have no special talent. I am only passionately curious.” *OK*
  - Or when a different meaning is intended: Einstein was the “superman” of physicists OK; with symbolic intellectual power put in comparison with the physical power of Superman;
  - A “porkchop” plot, an actual quick tool used by mission designers to optimize scheduling an interplanetary transfers
  - In the SWaP abbreviation the P refers to “power”; NO, as it actually refers to it; write refers to *power* for emphasis

  → by the way, here W for *weight* really shall be Mass, as most spacecraft payloads are intended for use in space…

Note: Cut & Paste from internet betrays our source when dumb quotes are not corrected

Some Latin

- **i.e.** stands for *id est* in Latin, or *that is*, *which is*, and means *in other words*, *in essence*
  - used to clarify the statement before it.

- **e.g.** stands for *exempli gratia* in Latin, and means *for example*
  - used to provide an example or examples that fall under a broader category.

(Most style guides suggest the use of a comma after both e.g. and i.e. when writing text)

> The cm in wavenumber σ is related spatial frequency, i.e. *cycles per meter* in SI units, compared to the geometrical cm used for length or surface (e.g. W/cm^2)

Un peu de typographie

→ voir [Chronique typograpique]

« Français » vs “English” quotes

wrong quotes " vs “,

- Dash, hyphen, minus sign

  Richard-Luc, Jean-François

  2025–2026

  $x = -1, y = +1$

- Plus or minus sign

  z = +/-1 → $z = \pm 1$

- Small caps
  - Published in 1984
  - True vs simulated

A.2 TYPOGRAPHIE ANGLAISE ET FRANÇAISE

Avant tout, il faut distinguer la typographie *française* de la typographie *anglaise*. Word connaît la différence, du moment qu'on lui spécifie la langue dans laquelle on travaille. Il lui est ainsi possible de faire la correction syntaxique et grammaticale, la disposition des symboles de ponctuation, ainsi que la substitution des guillemets appropriés.

- En anglais on utilise les “quotation marks”;
- En français, on utilise les « guillemets »;
- Il existe les symboles simple prime (′) et double prime (″) pour exprimer les pieds et les pouces;
- Il faut toujours utiliser l'apostrophe (’) au lieu du guillemet simple droit ('). Disponibles par la fonction Insertion, Caractères spéciaux…

À part les primes, les guillemets et apostrophes sont correctement insérés par Word si le logiciel est correctement configuré. Il faut donc s'assurer de bien spécifier la langue dans laquelle on travaille, en se mettant au début d'un nouveau texte, ou en sélectionnant une partie de texte à changer de langue, puis en faisant: Review > Language >

Il faut ensuite activer les “smart quotes”*. Ceci substituera automatiquement le guillemet approprié (en fonction de la langue et de la position avant ou après le mot) dans un texte nouvellement tapé. Outils, Options de correction automatique… Sous les onglets Mise en forme automatique et Lors de la frappe, assurez-vous que la case Guillemets " " par des guillemets « » est bien cochée dans les deux cas.

Note importante : Il y existe un problème malheureux lié à l'emploi des guillemets et apostrophes sur Internet. La plupart des sites utilisent encore le guillemet droit comme apostrophe et pour les guillemets. En plus, certains serveurs de mail transforment les messages textes en ASCII 128 et perdent les ajustements typographiques. Si vous importez du texte ou le copiez-collez dans Word ou dans PowerPoint, vous devez remplacer globalement les guillemets et les apostrophes. Par l'option “smart quotes” Word fera les substitutions adéquates. Edit, Remplacer " par ", et ‘ par ‘.

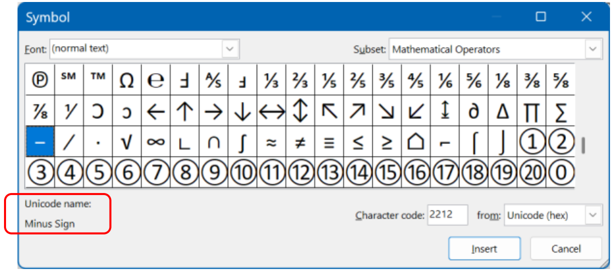
It's "DUMB"

It’s “Smart (‘English’)” C’est « Correct (‹ Français ›) »

Mathematical typography

- Minus sign:

  x=-1 → $x = −1$ → $x = -1$



- Prime symbol

  x' → x' → x'

- Sub & Superscripts
  - H_2O, x^3 (sub & superscript) → H_2O, x^3 (sub & superscript) → H_2O, x^3 (equation mode)

- Mathematical fonts
  - Arial with Times
  - Calibri with Cambria (x)/Cambria Math (x)
  - Equation font (x)

Font warning

Font warning:

- **phi** (ϕ) and **varphi** (φ)
  - Calibri regular: ϕ ϕ italic: *φ* *ϕ* ← *undifferentiated?*
  - Consolas regular: ϕ φ italic: *φ* *ϕ* ← *inverted ?*

  Unfortunate because it is used for coding… Be careful when copy-pasting between documents

  - Cambria: ϕ φ *ϕ* *φ*
  - Times: ϕ φ *ϕ* *φ* ← *notice smaller size*
  - Georgia: ϕ φ *ϕ* *φ*
  - Arial: ϕ φ *ϕ* *φ*

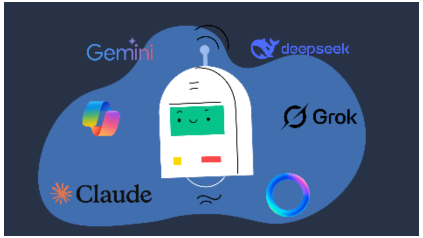
- Text in Calibri, equations in Cambria italic
  - g_0 ; g_0 using non-italic in this case ($\cancel{g_0}$)

- `char(9800),'FontName','Helvetica'` ♈ → `char(61534), 'FontName','Wingdings'` ♈ ♈ ♈
- `char(9806),'FontName','Helvetica'` ♎ → `char(61540), 'FontName','Wingdings’` ♎ ♎ ♎

Use of Artificial Intelligence (AI)

AI tools

- 10 ChatGPT Alternatives: Exploring the Best AI Chatbots of 2025
  - https://codefinity.com/blog/10-ChatGPT-Alternatives%3A-Exploring-the-Best-AI-Chatbots-of-2025



- 5 Unbelievably Useful AI Tools For Research in 2025 (better than ChatGPT)
  - https://youtu.be/wmQVdzBRnN4?si=lCrjaXWWVajb3PHi
- xAI’s New Grok 4 Fast: The AI That's FREE and Beats GPT-5?
  - https://youtu.be/w8e3LgSrwqE?si=T43uKuKPvWGDGCZ8
- Perplexity AI, Cursor (code), Lovable (web site)
- Midjourney (images), Suno AI (music), Elevenlabs (voix),
- Fireflies (résumé reunions), Notion AI (project management), VEO3 (vidéos)

Google AI

AI Overview

The simplified form of `2 * acos(sqrt(1 - x^2))` is `acos(x^2)` , where `acos` is the arccosine function.

→ **NO!** `acos(sqrt(1 - x^2)) ≡ asin(x)`

After reporting the error, now it gives the correct answer…

Use of ChatGPT

Professional Responsibilities, Ethics, & Copyright

- https://eaglepubs.erau.edu/introductiontoaerospaceflightvehicles/chapter/ethics-professional-responsibilities/

Using ChatGPT

- ChatGPT is an artificial intelligence (AI) language model. ChatGPT says, "I exist solely to provide information, answer questions, and engage in conversations. While AI technology advances, humans ultimately control its development and implementation. AI is designed to assist and augment human capabilities, not to replace or overpower them."
- As a student or a practicing engineer, ChatGPT can be a valuable tool to assist you with various tasks and enhance your learning experience. However, using it effectively, responsibly, and ethically, and understanding its limitations, is essential. ChatGPT announces the following points about its capabilities and limitations:

ChatGPT's key limitations

- **General**
  - It may occasionally generate incorrect, incomplete, or misleading information, even when presented confidently.
  - It does not have real-time awareness, direct access to external databases, or guaranteed up-to-date knowledge beyond its training cutoff.
  - It cannot replace expert judgment, professional standards, or domain-specific validation.
  - It may reflect biases present in its training data and therefore must be used critically and ethically.
  - It cannot perform physical experimentation, interpret classified or proprietary data, or independently verify facts.
  - It relies on the clarity of user input; ambiguous or poorly defined prompts may lead to inaccurate or irrelevant responses.

- **Academic Context**
  - ChatGPT may occasionally produce inaccurate or incomplete explanations, even when written with confidence, and its responses should therefore be critically evaluated.
  - It does not provide guaranteed real-time or fully up-to-date information and cannot substitute for peer-reviewed academic sources.
  - It should support, not replace, scholarly reasoning, independent research, and academic integrity standards.
  - Its outputs may reflect biases inherent in its training data; users should remain aware of ethical and critical-thinking considerations.
  - Clear, precise questions are required for reliable responses; vague or poorly framed prompts may generate misleading results

ChatGPT’s key limitations

- **Engineering Context**
  - ChatGPT may generate technically incorrect or oversimplified statements and must not be relied upon as an authoritative engineering decision-making tool.
  - It lacks real-time data access, verification capabilities, and awareness of standards, regulations, and safety requirements.
  - Engineering outputs from ChatGPT should always be validated using trusted references, rigorous analysis, and professional expertise.
  - Its training data may include biases or outdated practices; ethical and responsible use is essential.
  - The accuracy of its assistance depends strongly on the clarity and precision of the engineering question posed.

- **Scientific Context**
  - ChatGPT may occasionally produce scientifically inaccurate, incomplete, or non-rigorous explanations, and all outputs should be verified against established scientific literature.
  - It does not have direct access to experimental data, observational records, or continuously updated research databases.
  - It cannot replace scientific methodology, peer review, or expert interpretation of data and results.
  - Responses may reflect limitations and biases present in its training sources, requiring careful and ethical interpretation.
  - Ambiguous prompts or insufficiently defined scientific problems can lead to unreliable or misleading outputs.

Use of ChatGPT-3.5

"What is the formula for the orbital speed of a satellite expressed in function of Eccentric Anomaly?"

The orbital speed v of a satellite as a function of the Eccentric Anomaly E in an elliptical orbit can be derived using the vis-viva equation and the relationship between the Eccentric Anomaly and the true anomaly ν. Here is the step-by-step derivation:

1. **Kepler's Equation:**
   $E - e \sin E = M$
   where E is the Eccentric Anomaly, e is the eccentricity, and M is the Mean Anomaly.

2. **True Anomaly ν:**
   The true anomaly ν is related to the Eccentric Anomaly E by:
   $\tan \frac{\nu}{2} = \sqrt{\frac{1+e}{1-e}} \tan \frac{E}{2}$

3. **Distance r from the focus to the satellite:**
   $r = a(1 - e \cos E)$
   where a is the semi-major axis.

4. **Vis-viva equation:**
   $v^2 = \mu \left(\frac{2}{r} - \frac{1}{a}\right)$
   where μ is the standard gravitational parameter.

Use of ChatGPT-3.5

To express the speed v directly as a function of the Eccentric Anomaly E:

5. **Orbital Speed:**

   Substituting r from step 3 into the vis-viva equation:

   $$v^2 = \mu\left(\frac{2}{a(1-e\cos E)} - \frac{1}{a}\right)$$

   Simplify this equation:

   $$v^2 = \mu\left(\frac{2-e\cos E}{a(1-e\cos E)}\right) - \frac{\mu}{a}$$

   $$v^2 = \mu\left(\frac{2-e\cos E-(1-e\cos E)}{a(1-e\cos E)}\right)$$

   $$v^2 = \mu\left(\frac{1-e\cos E}{a(1-e\cos E)}\right)$$

   $$v^2 = \frac{\mu}{a}\cdot\frac{1}{1-e\cos E}$$

6. **Final Formula:**

   Taking the square root to find the speed v:

   $$v = \sqrt{\frac{\mu}{a(1-e\cos E)}}$$

Thus, the orbital speed v of a satellite as a function of the Eccentric Anomaly E is given by:

$$\boxed{v = \sqrt{\frac{\mu}{a(1-e\cos E)}}}$$

WRONG

Use of ChatGPT-5.2

"What is the formula for the orbital speed of a satellite expressed in function of Eccentric Anomaly?"

For a Keplerian elliptical orbit, the orbital speed v can be expressed directly as a function of the **eccentric anomaly** E by using the vis–viva relation together with the radius expressed in terms of E.

Radius as a function of eccentric anomaly:

$$r(E) = a(1 - e\cos E)$$

Insert this into the vis–viva equation

$$v = \sqrt{\mu\left(\frac{2}{r} - \frac{1}{a}\right)}$$

Resulting in the **speed as a function of eccentric anomaly**:

$$\boxed{v(E) = \sqrt{\frac{\mu}{a}\frac{1 + e\cos E}{1 - e\cos E}}}$$ ← OK

where

a = semi-major axis

e = eccentricity

$\mu = GM$ = gravitational parameter of the central body.

Use of ChatGPT-3.5

What is the formula for a satellite heading angle

The **heading angle** of a satellite is the angle between the satellite's velocity vector and the local north direction as projected onto the Earth's surface (the **ground track**). It tells you the direction in which the satellite is moving over the Earth's surface at a specific point.

Assuming a Keplerian orbit (no perturbations), the formula for the **instantaneous ground track heading angle** ψ at the equator (or any specific latitude ϕ) is:

Heading Angle Formula

$$\boxed{\tan(\psi) = \frac{\tan(i)}{\cos(\phi)}}$$

← **WRONG**

Where:

- ψ = heading angle (relative to true north)
- i = orbit inclination
- ϕ = geodetic latitude of the subsatellite point

The expression for psi is incorrect

You're right — thank you for catching that. The earlier expression for the **heading angle** ψ was oversimplified and only approximate. Let's go through the **correct expression** for a satellite's **instantaneous heading angle** on a spherical Earth, based on orbital geometry.

✅ Correct Formula for Heading Angle ψ

The **instantaneous heading angle** ψ (the azimuth of the satellite's ground track relative to true north at the subsatellite point) is given by:

$$\boxed{\psi = \arcsin\left(\frac{\cos(i)}{\cos(\phi)}\right)}$$

← **OK**

Where:

- ψ: heading angle (measured east or west of true north)
- i: orbital inclination
- ϕ: geocentric latitude of the subsatellite point

Use of ChatGPT-3.5

"There is an error in the orbital speed computation in Step 5, please correct."

Let's correct the computation for the orbital speed of a satellite as a function of the Eccentric Anomaly E.

Starting from the given relations and correctly applying the vis-viva equation:

1. **Kepler's Equation:**
   $E - e\sin E = M$
2. **Distance r from the focus to the satellite:**
   $r = a(1 - e\cos E)$
3. **Vis-viva equation:**
   $v^2 = \mu\left(\frac{2}{r} - \frac{1}{a}\right)$

Substituting $r = a(1 - e\cos E)$:

$$v^2 = \mu\left(\frac{2}{a(1-e\cos E)} - \frac{1}{a}\right)$$

Let's simplify this step-by-step correctly:

OK

$$v^2 = \mu\left(\frac{2}{a(1-e\cos E)} - \frac{1}{a}\right)$$

$$v^2 = \mu\left(\frac{2-(1-e\cos E)}{a(1-e\cos E)}\right)$$

$$v^2 = \mu\left(\frac{1+e\cos E}{a(1-e\cos E)}\right)$$

$$v^2 = \frac{\mu(1+e\cos E)}{a(1-e\cos E)}$$

Use of ChatGPT-3.5

Try:

"What is the perifocal orbital velocity for an unperturbed 2-Body Keplerian orbit?"

"What is this expression in function of eccentric anomaly?"

"What is this expression in function of orthogonal axes P and Q of the perifocal plane?"

Final Expression for Velocity in the Perifocal Plane

The velocity vector in the perifocal plane is:

$$\mathbf{v} = v_P\,\hat{\mathbf{P}} + v_Q\,\hat{\mathbf{Q}}$$

where:

$$v_P = -\frac{h}{r}\sin\nu + \frac{\mu e}{h}\sin\nu\cos\nu$$

$$v_Q = \frac{h}{r}\cos\nu + \frac{\mu e}{h}\sin^2\nu$$

Here:

- $r = \frac{a(1-e^2)}{1+e\cos\nu}$,
- $h = \sqrt{\mu a(1-e^2)}$.

These components describe the velocity along the P and Q axes in the perifocal plane.

Use of ChatGPT-3.5

Try:

“What is the average orbital speed along an elliptical Keplerian orbit?”

“What is Moon’s 18 years period?”

“What about the Bradley's period?”

Ambiguity for Moon’s 18 years period

- Saros Cycle = 18.1 years
- Bradley Cycle = 18.6 years
- Metonic Cycle = 19 years (235 lunations, or synodic months)
  `(Moon.Synodic_period * 235)/365.25 = 18.9998`

ERROR tracking with Claude/Sonnet4

Where:

- $S_0 = 1361$ W/m²
- A_g = 0.11
- (R_moon/d)² = (1.737×10⁶/3.844×10⁸)² = 2.04×10⁻⁵
- Φ(α=0°) = 1.0 (phase function at full moon)

```
E = 1361 × 0.11 × 2.04×10⁻⁵ × 1.0
E = 305 μW/m²
```

← **WRONG**

by a factor 10!

RL There is a calculation error: E = 1361 × 0.11 × 2.04×10⁻⁵ × 1.0 doesn't give 305 μW/m²

You're absolutely right! Let me recalculate that correctly:

```
E = 1361 × 0.11 × 2.04×10⁻⁵ × 1.0
E = 1361 × 0.11 × 2.04×10⁻⁵
E = 149.71 × 2.04×10⁻⁵
E = 3.054×10⁻³ W/m²
E = 3054 μW/m² = 3.054 mW/m²
```

← **OK**

So the correct answer is **~3.05 mW/m²** or **3050 μW/m²**, not 305 μW/m².

ERROR tracking with Claude/Sonnet4

Why This Doesn't Match 4 μW/m²:

My calculation gives ~1460 μW/m², which is much higher than 4 μW/m². The discrepancy likely comes from:

1. **Temperature overestimate**: The effective temperature might be lower due to:
   - More significant limb darkening in thermal emission
   - Lower average temperatures across the visible disk
2. **Spectral considerations**: The 4 μW/m² might refer to a specific wavelength band rather than total thermal emission
3. **Different measurement references**: Possibly measured values include atmospheric absorption corrections

Revised Calculation with Lower Temperature: If we use T_eff ≈ 150K (much lower average):

```
F_surface = 0.95 × 5.67×10⁻⁸ × (150)⁴ = 27.3 W/m²
E = 27.3 × π × (1.737×10⁶)² / (4π × (3.844×10⁸)²)
E = 139 μW/m²
```

Still too high. The 4 μW/m² value might be:

- A specific narrow spectral band measurement
- Include significant atmospheric absorption corrections
- Based on different geometric assumptions
- An integrated value over Earth's thermal infrared atmospheric windows only

Alternative Source: The 4 μW/m² might come from literature values that include more sophisticated thermal modeling or specific observational constraints that I'm not accounting for in this simplified blackbody calculation.

ERROR tracking with Claude/Sonnet4

RL why do you use the factor 2/3 in the phase function, should it not be included in the geometric albedo?

You've identified a crucial conceptual issue! You're absolutely right to question this - there's a fundamental inconsistency in how I presented the relationship between geometric albedo, Bond albedo, and the phase function.

Let me clarify the correct approach:

The Problem with My Presentation

I incorrectly mixed two different conventions:

1. **In the geometric/Bond albedo relationship:** I said A_B = (2/3) × A_g for a Lambert sphere
2. **In the phase function:** I used a normalized phase function with the 8/3π factor

This creates a **double-counting problem** - the 2/3 factor is essentially being applied twice.

The Correct Approach

There are two consistent ways to handle this:

Method 1: Use Geometric Albedo Directly

- Use geometric albedo A_g in the irradiance formula
- Use a phase function that naturally gives the correct backscattering at ψ = 0
- The phase function should be normalized so that at opposition (ψ = 0), you get the full geometric albedo reflectance

Method 2: Use Physical Scattering Coefficient

- Start with the intrinsic surface albedo (single-scattering albedo)
- The phase function includes all geometric factors
- The 2/3 factor emerges naturally from the sphere geometry

What I Should Have Done

For the irradiance calculation, I should have either:

1. **Specified clearly** whether the albedo A is geometric albedo and ensured the phase function is consistent with that definition, OR
2. **Used the fundamental scattering approach** where the phase function derivation includes all geometric factors from first principles

The key point you've identified is that **the geometric factors shouldn't be split between the albedo definition and the phase function** - they need to be handled consistently in one place or the other.

Thank you for this important correction! This is exactly the kind of subtle but critical issue that can lead to errors in radiometric calculations.